

程式人十分鐘系列



用十分鐘瞭解 Tensorflow.js

(Google 的 JavaScript 深度學習套件)

陳鍾誠

金門大學資訊工程系

2018 年 4 月 24 日

話說

- 這兩年 Python 快速崛起
- 原因主要和《深度學習》有關...

特別是

- Tensorflow 框架只支援 Python 語言做機器學習訓練...
- 其他的語言，像是 Java, Go, JavaScript 只能使用 Python 訓練好的模型，而無法撰寫訓練程式 ...

這對以 JavaScript 為主的我

- 實在是非常不方便的....

終於

- 在 2018 年四月

Google 的 Nikhil Thorat 等人

釋出了 Tensorflow.js

Tensorflow.js

- 不只可以載入 Python 訓練好的模型，也可以直接撰寫訓練程式...

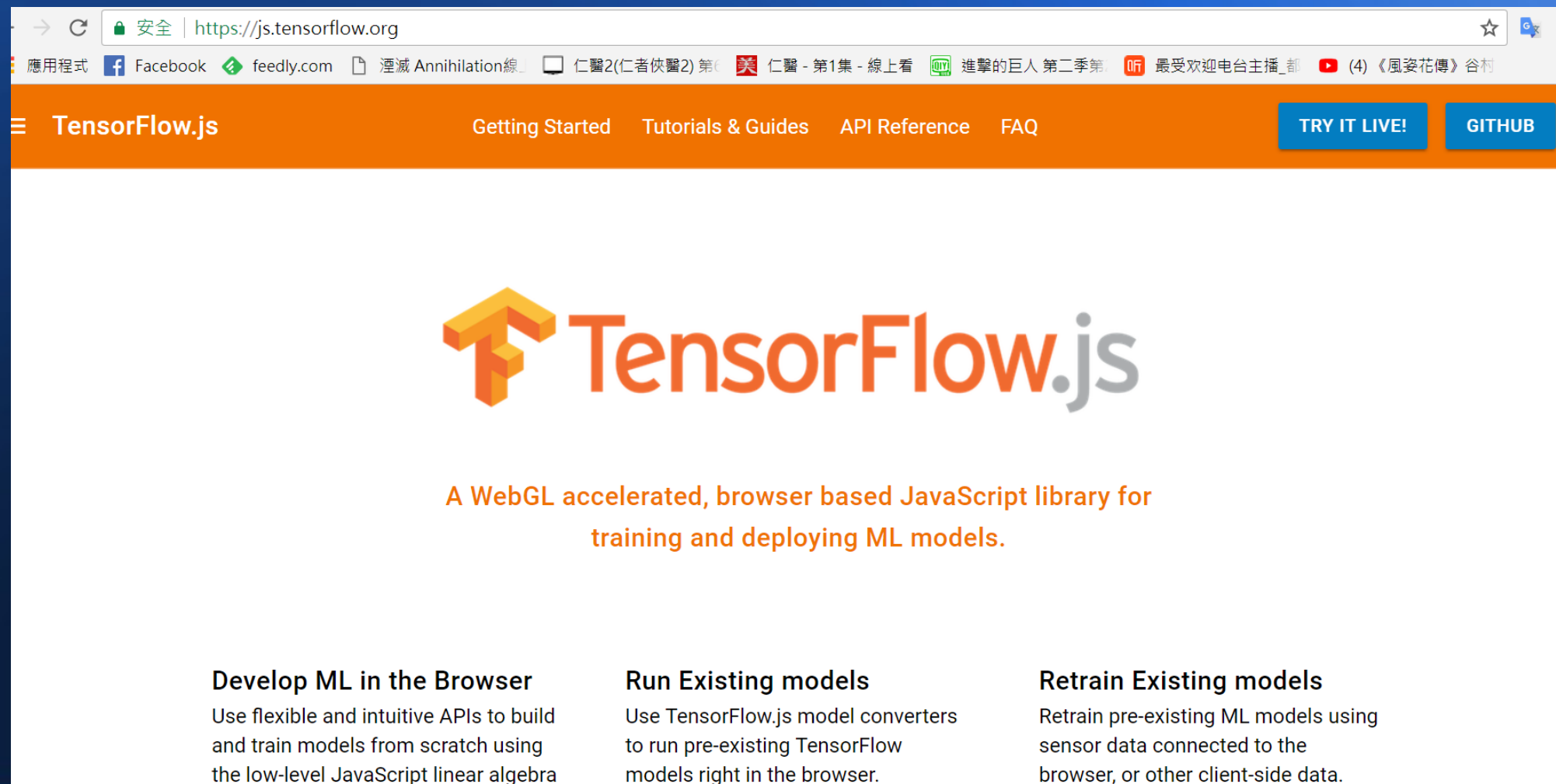
這對我來說

- 無疑是一個非常令人興奮的消息...

所以我決定

- 立馬開始使用 Tensorflow.js ...

首先來到 Tensorflow.js 的官網



The screenshot shows the TensorFlow.js website in a web browser. The browser's address bar displays 'https://js.tensorflow.org' with a green lock icon and the text '安全'. The page features an orange navigation bar with the 'TensorFlow.js' logo on the left and links for 'Getting Started', 'Tutorials & Guides', 'API Reference', and 'FAQ' in the center. On the right side of the navigation bar are two blue buttons: 'TRY IT LIVE!' and 'GITHUB'. Below the navigation bar, the main content area has a white background. It features the TensorFlow.js logo, which consists of an orange 3D cube icon followed by the text 'TensorFlow.js'. Below the logo, a line of orange text reads: 'A WebGL accelerated, browser based JavaScript library for training and deploying ML models.' At the bottom of the page, there are three columns of text. The first column is titled 'Develop ML in the Browser' and describes using flexible APIs to build and train models from scratch. The second column is titled 'Run Existing models' and describes using model converters to run pre-existing TensorFlow models in the browser. The third column is titled 'Retrain Existing models' and describes retraining pre-existing ML models using sensor data connected to the browser or other client-side data.

→ ↻ 安全 | <https://js.tensorflow.org> ☆

應用程式 Facebook feedly.com 湮滅 Annihilation線 仁醫2(仁者俠醫2)第 仁醫 - 第1集 - 線上看 進擊的巨人 第二季第 最歡迎電台主播_都 (4)《風姿花傳》谷村

TensorFlow.js Getting Started Tutorials & Guides API Reference FAQ TRY IT LIVE! GITHUB



A WebGL accelerated, browser based JavaScript library for training and deploying ML models.

Develop ML in the Browser Use flexible and intuitive APIs to build and train models from scratch using the low-level JavaScript linear algebra	Run Existing models Use TensorFlow.js model converters to run pre-existing TensorFlow models right in the browser.	Retrain Existing models Retrain pre-existing ML models using sensor data connected to the browser, or other client-side data.
--	--	---

<https://js.tensorflow.org/>

向下捲到 Getting Started (via Script Tag 這一段)

安全 | <https://js.tensorflow.org>

via Script Tag

Add the following code to an HTML file:

```
<html>
<head>
  <!-- Load TensorFlow.js -->
  <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.9.0"> </script>

  <!-- Place your code in the script tag below. You can also use an external .js file -->
  <script>
    // Notice there is no 'import' statement. 'tf' is available on the index-page
    // because of the script tag above.

    // Define a model for linear regression.
    const model = tf.sequential();
    model.add(tf.layers.dense({units: 1, inputShape: [1]}));

    // Prepare the model for training: Specify the loss and the optimizer.
    model.compile({loss: 'meanSquaredError', optimizer: 'sgd'});

    // Generate some synthetic data for training.
    const xs = tf.tensor2d([1, 2, 3, 4], [4, 1]);
    const ys = tf.tensor2d([1, 3, 5, 7], [4, 1]);

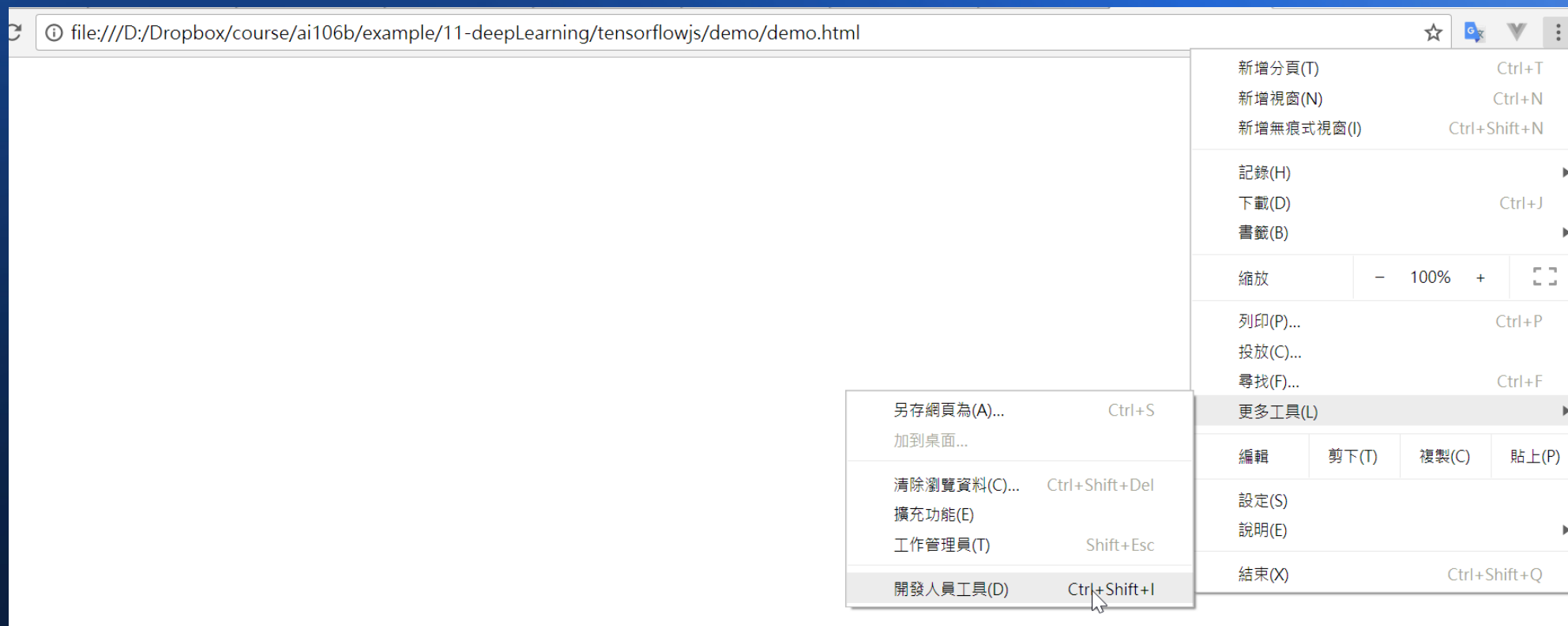
    // Train the model using the data.
    model.fit(xs, ys).then(() => {
      // Use the model to do inference on a data point the model hasn't seen before:
      // Open the browser devtools to see the output
      model.predict(tf.tensor2d([5], [1, 1])).print();
    });
  </script>
```

剪貼該程式後存成 demo.html 檔案

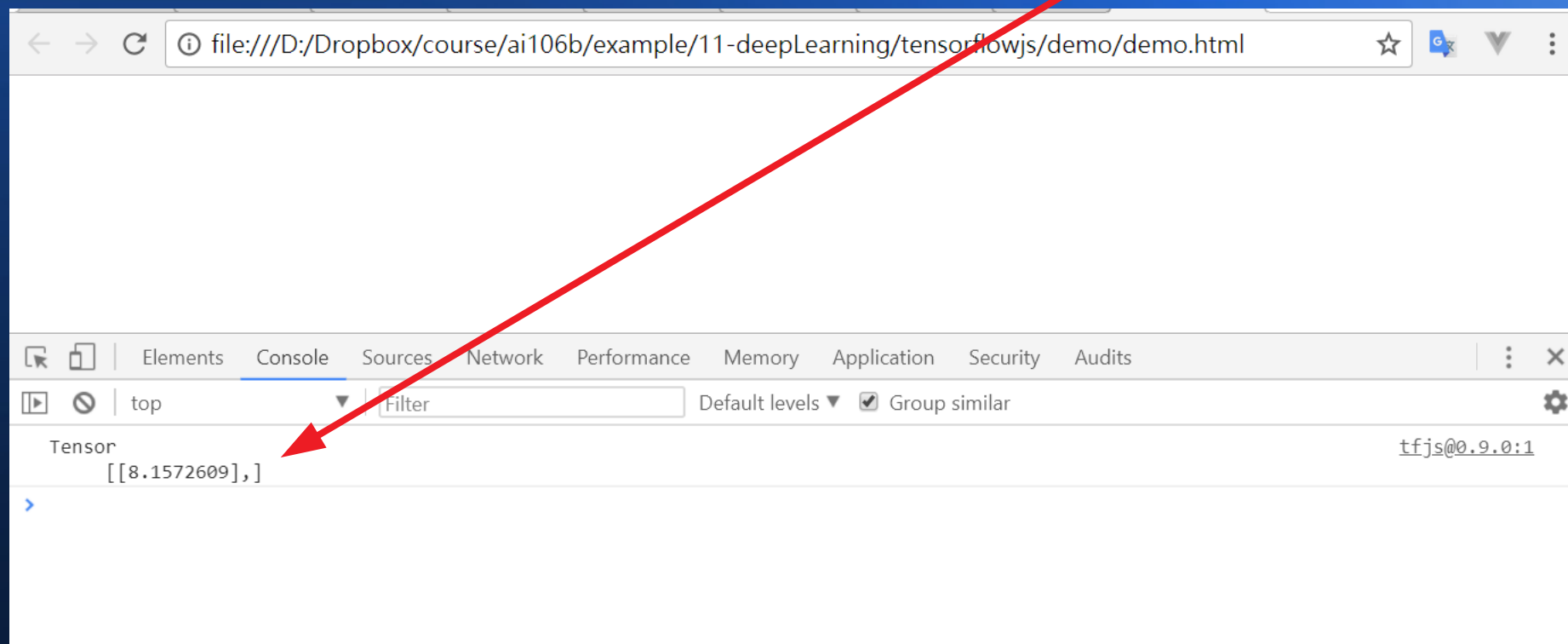
<> demo.html x

```
1 <html>
2   <head>
3     <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.9.0"></script>
4     <script>
5       const model = tf.sequential();
6       model.add(tf.layers.dense({units: 1, inputShape: [1]}));
7       model.compile({loss: 'meanSquaredError', optimizer: 'sgd'});
8       // Generate some synthetic data for training.
9       const xs = tf.tensor2d([1, 2, 3, 4], [4, 1]);
10      const ys = tf.tensor2d([1, 3, 5, 7], [4, 1]);
11      // Train the model using the data.
12      model.fit(xs, ys).then(() => {
13        // Use the model to do inference on a data point the model hasn't seen before:
14        // Open the browser devtools to see the output
15        model.predict(tf.tensor2d([5], [1, 1])).print();
16      });
17    </script>
18  </head>
19  <body>
20  </body>
21 </html>
```

打開該檔後選 《更多工具 / 開發人員工具》



然後我們可以看到程式的輸出



仔細看一下程式

<> demo.html x

```
1 <html>
2   <head>
3     <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.9.0"></script>
4     <script>
5       const model = tf.sequential();
6       model.add(tf.layers.dense({units: 1, inputShape: [1]}));
7       model.compile({loss: 'meanSquaredError', optimizer: 'sgd'});
8       // Generate some synthetic data for training.
9       const xs = tf.tensor2d([1, 2, 3, 4], [4, 1]);
10      const ys = tf.tensor2d([1, 3, 5, 7], [4, 1]);
11      // Train the model using the data.
12      model.fit(xs, ys).then(() => {
13        // Use the model to do inference on a data point the model hasn't seen before:
14        // Open the browser devtools to see the output
15        model.predict(tf.tensor2d([5], [1, 1])).print();
16      });
17    </script>
18  </head>
19  <body>
20  </body>
21 </html>
```

查查其中 `tf.sequential` 的說明

[`tf.sequential`](#) (config?)

function

[source](#)

Creates a [`tf.Sequential`](#) model. A sequential model is any model where the outputs of one layer are the inputs to the next layer, i.e. the model topology is a simple 'stack' of layers, with no branching or skipping.

This means that the first layer passed to a Sequential model should have a defined input shape. What that means is that it should have received an `inputShape` or `batchInputShape` argument, or for some type of layers (recurrent, Dense...) an `inputDim` argument.

The key difference between [`tf.model\(\)`](#) and [`tf.sequential\(\)`](#) is that [`tf.sequential\(\)`](#) is less generic, supporting only a linear stack of layers. [`tf.model\(\)`](#) is more generic and supports an arbitrary graph (without cycles) of layers.

Examples:

```
const model = tf.sequential();

// First layer must have an input shape defined.
model.add(tf.layers.dense({units: 32, inputShape: [50]}));
// Afterwards, TF.js does automatic shape inference.
model.add(tf.layers.dense({units: 4}));

// Inspect the inferred shape of the model's output, which equals
// `[null, 4]`. The 1st dimension is the undetermined batch dimension; the
// 2nd is the output size of the model's last layer.
console.log(JSON.stringify(model.outputs[0].shape));
```

所以 tf.sequential 代表

- 將神經網路一個接一個串接起來

<> demo.html x

```
1 <html>
2 <<head>
3 <<<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.9.0"></script>
4 <<<script>
5 <<<<const model = tf.sequential();
6 <<<<model.add(tf.layers.dense({units: 1, inputShape: [1]}));
7 <<<<model.compile({loss: 'meanSquaredError', optimizer: 'sgd'});
8 <<<<// Generate some synthetic data for training.
9 <<<<const xs = tf.tensor2d([1, 2, 3, 4], [4, 1]);
10 <<<<const ys = tf.tensor2d([1, 3, 5, 7], [4, 1]);
11 <<<<// Train the model using the data.
12 <<<<model.fit(xs, ys).then(() => {
13 <<<<<// Use the model to do inference on a data point the model hasn't seen before:
14 <<<<<// Open the browser devtools to see the output
15 <<<<<model.predict(tf.tensor2d([5], [1, 1])).print();
16 <<<<<});
17 <<<</script>
18 <<</head>
19 <<<body>
20 <<</body>
21 </html>
```


這樣我就看懂了...

- 這個程式只有一層 dense 網路，然後用隨機梯度下降法，對 meanSquaredError 這個能量函數進行優化！

<> demo.html x

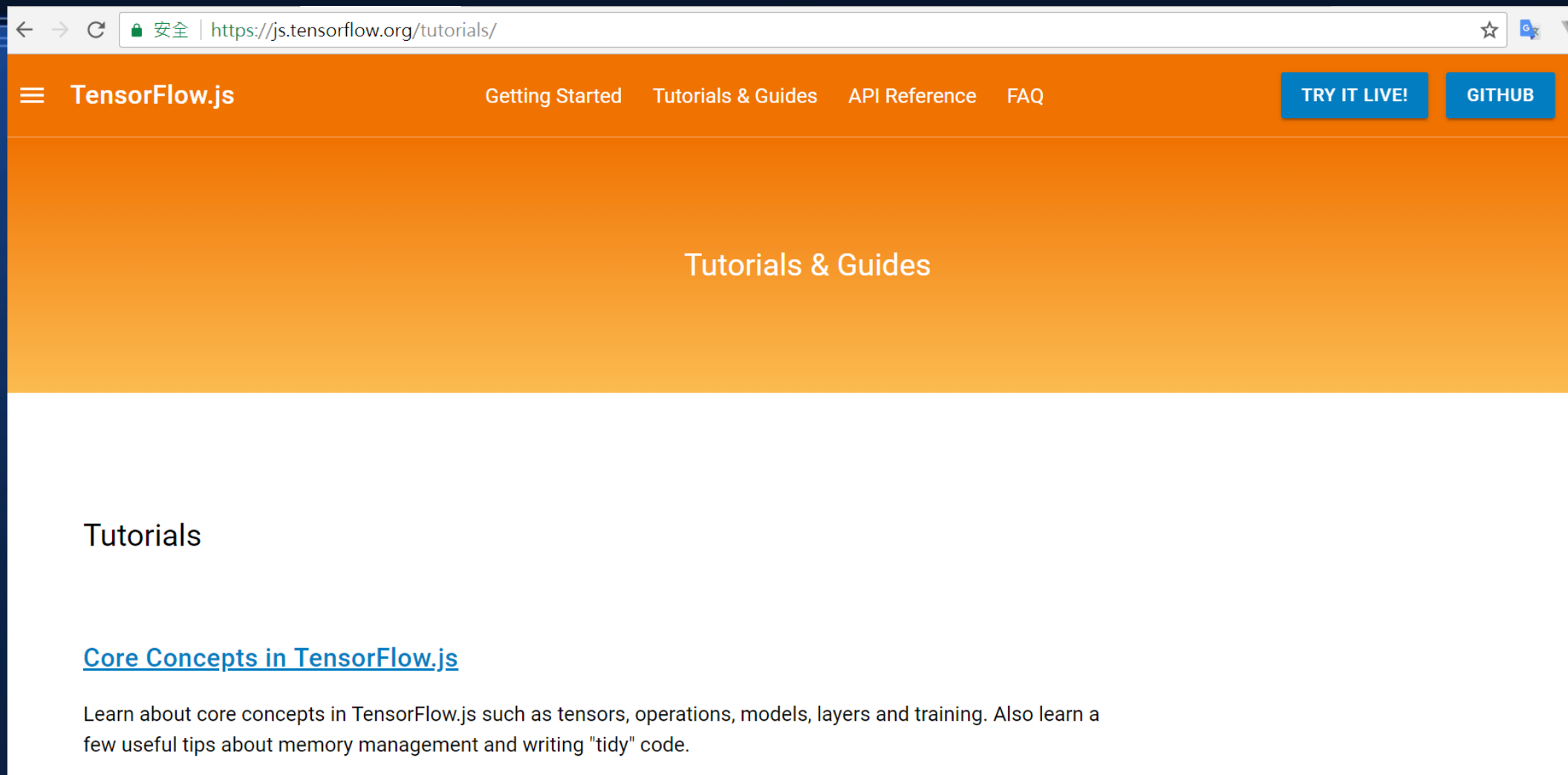
```
1 <html>
2 <head>
3   <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.9.0"></script>
4   <script>
5     const model = tf.sequential();
6     model.add(tf.layers.dense({units: 1, inputShape: [1]}));
7     model.compile({loss: 'meanSquaredError', optimizer: 'sgd'});
8     // Generate some synthetic data for training.
9     const xs = tf.tensor2d([1, 2, 3, 4], [4, 1]);
10    const ys = tf.tensor2d([1, 3, 5, 7], [4, 1]);
11    // Train the model using the data.
12    model.fit(xs, ys).then(() => {
13      // Use the model to do inference on a data point the model hasn't seen before:
14      // Open the browser devtools to see the output
15      model.predict(tf.tensor2d([5], [1, 1])).print();
16    });
17  </script>
18 </head>
19 <body>
20 </body>
21 </html>
```

接著給定訓練的輸入與輸出

然後開始訓練，訓練好後就進行預測
看看 $x=5$ 時，預測的 y 會是多少？

有了這個範例當基礎

讓我們進一步看看 Tutorials & Guides

A screenshot of a web browser displaying the TensorFlow.js Tutorials & Guides page. The browser's address bar shows the URL 'https://js.tensorflow.org/tutorials/'. The page has an orange header with the TensorFlow.js logo on the left and navigation links for 'Getting Started', 'Tutorials & Guides', 'API Reference', and 'FAQ' in the center. On the right side of the header are two blue buttons: 'TRY IT LIVE!' and 'GITHUB'. Below the header, the main content area has a large orange banner with the text 'Tutorials & Guides' in white. Underneath this banner, the word 'Tutorials' is displayed in a large, dark font. Below 'Tutorials', there is a blue link titled 'Core Concepts in TensorFlow.js'. At the bottom of the visible content, a paragraph of text describes the core concepts and provides additional learning resources.

← → ↻ 安全 | https://js.tensorflow.org/tutorials/ ☆

≡ TensorFlow.js Getting Started Tutorials & Guides API Reference FAQ TRY IT LIVE! GITHUB

Tutorials & Guides

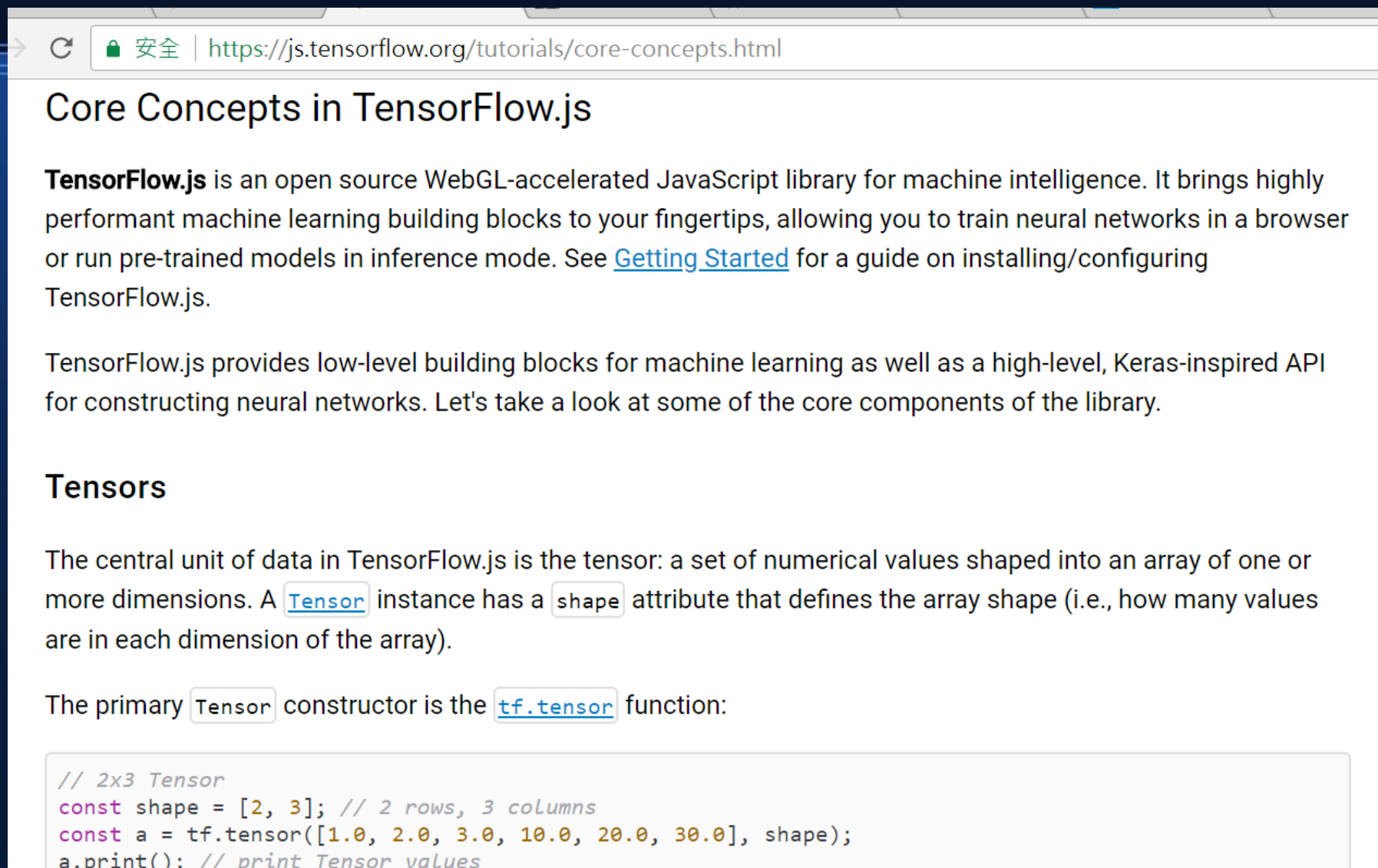
Tutorials

[Core Concepts in TensorFlow.js](#)

Learn about core concepts in TensorFlow.js such as tensors, operations, models, layers and training. Also learn a few useful tips about memory management and writing "tidy" code.

<https://js.tensorflow.org/tutorials/>

先看看 core-concepts



The screenshot shows a web browser window with the URL <https://js.tensorflow.org/tutorials/core-concepts.html>. The page title is "Core Concepts in TensorFlow.js". The main text describes TensorFlow.js as an open source WebGL-accelerated JavaScript library for machine intelligence. It mentions that it provides low-level building blocks for machine learning as well as a high-level, Keras-inspired API. The section "Tensors" explains that the central unit of data is the tensor, which is a set of numerical values shaped into an array of one or more dimensions. It also mentions that the primary `Tensor` constructor is the `tf.tensor` function. At the bottom, there is a code block showing how to create a 2x3 tensor and print its values.

Core Concepts in TensorFlow.js

TensorFlow.js is an open source WebGL-accelerated JavaScript library for machine intelligence. It brings highly performant machine learning building blocks to your fingertips, allowing you to train neural networks in a browser or run pre-trained models in inference mode. See [Getting Started](#) for a guide on installing/configuring TensorFlow.js.

TensorFlow.js provides low-level building blocks for machine learning as well as a high-level, Keras-inspired API for constructing neural networks. Let's take a look at some of the core components of the library.

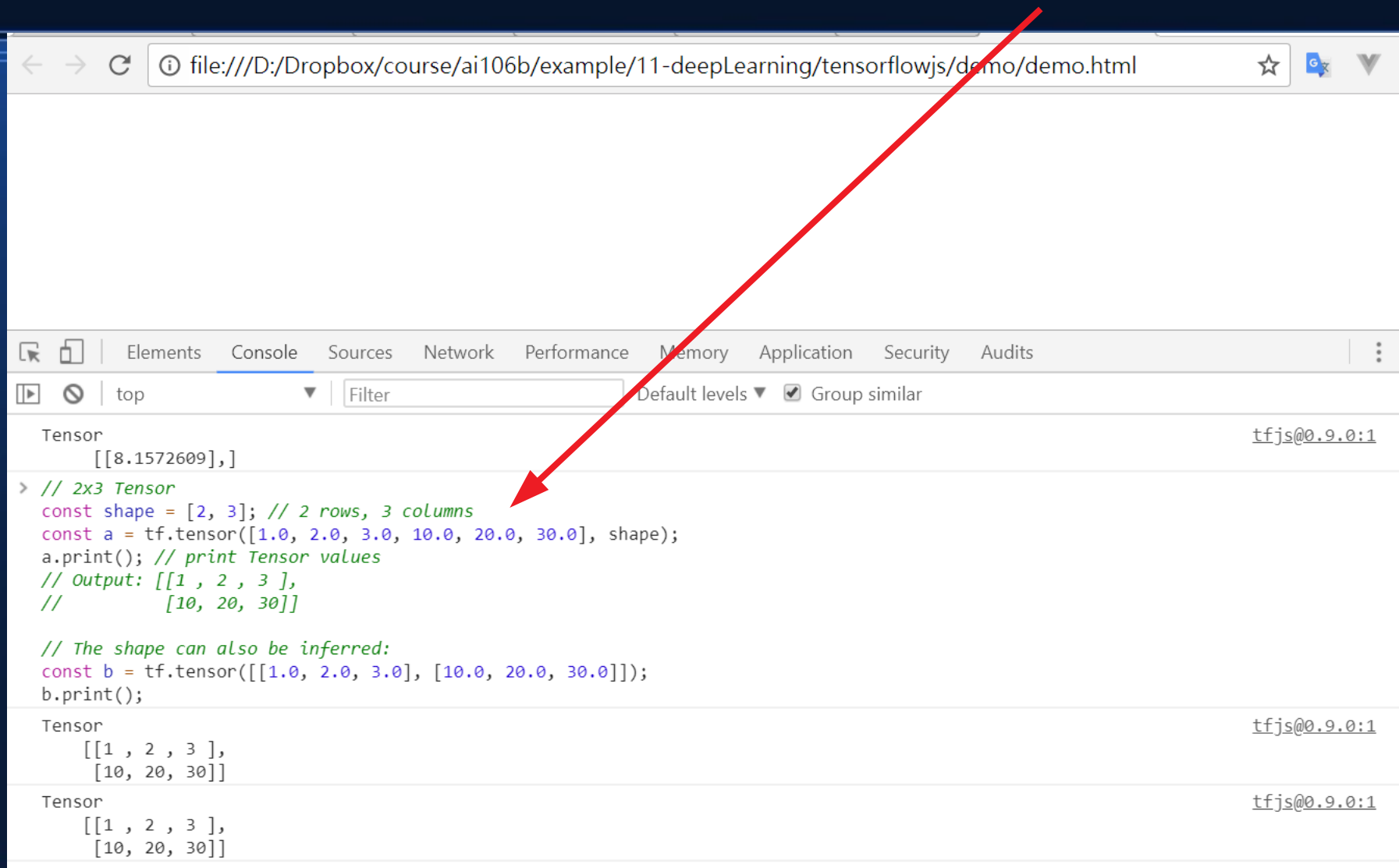
Tensors

The central unit of data in TensorFlow.js is the tensor: a set of numerical values shaped into an array of one or more dimensions. A `Tensor` instance has a `shape` attribute that defines the array shape (i.e., how many values are in each dimension of the array).

The primary `Tensor` constructor is the `tf.tensor` function:

```
// 2x3 Tensor
const shape = [2, 3]; // 2 rows, 3 columns
const a = tf.tensor([1.0, 2.0, 3.0, 10.0, 20.0, 30.0], shape);
a.print(); // print Tensor values
```

我們可以利用剛剛 demo.html 的開發人員工具來執行這些指令



但是必須先具備一點

- 神經網路與深度學習的基本概念
- 那就是 tensor 張量 ...

我們在中學數學裏學過向量與矩陣

- 我們可以把矩陣視為《2 維的向量》
- 那麼一般向量當然是《1 維的向量》
- 接著把單一數量視為《0 維的向量》

而張量 **tensor** 這個詞

- 用來泛指 $0, 1, 2, 3, 4, 5 \dots$ 等各種維度的向量

所以在神經網路的世界裏

- 幾乎一切都是張量 ...

所有的運算

- 也都是針對張量進行的

神經網路的程式

- 只要安排好
 - 張量如何流進去？
 - 經過甚麼運算？
 - 然後流出到哪裡 ...

就差不多完成了

這就是 Tensorflow 這個名詞的來源

- 也就是《張量流》...

深度學習除了張量之外

- 剩下的都是優化問題
- 可以交給《梯度下降法與反傳遞演算法》處理
- 這些在 tensorflow.js 裏都做好了！

所以學 Tensorflow. js

第一步就是要學張量的宣告與操作

The primary `Tensor` constructor is the `tf.tensor` function:

```
// 2x3 Tensor
const shape = [2, 3]; // 2 rows, 3 columns
const a = tf.tensor([1.0, 2.0, 3.0, 10.0, 20.0, 30.0], shape);
a.print(); // print Tensor values
// Output: [[1 , 2 , 3 ],
//          [10, 20, 30]]

// The shape can also be inferred:
const b = tf.tensor([[1.0, 2.0, 3.0], [10.0, 20.0, 30.0]]);
b.print();
// Output: [[1 , 2 , 3 ],
//          [10, 20, 30]]
```

However, for constructing low-rank tensors, we recommend using the following functions to enhance code readability: `tf.scalar`, `tf.tensor1d`, `tf.tensor2d`, `tf.tensor3d` and `tf.tensor4d`.

還有張量的初始化等動作

The following example creates an identical tensor to the one above using `tf.tensor2d`:

```
const c = tf.tensor2d([[1.0, 2.0, 3.0], [10.0, 20.0, 30.0]]);  
c.print();  
// Output: [[1 , 2 , 3 ],  
//          [10, 20, 30]]
```

TensorFlow.js also provides convenience functions for creating tensors with all values set to 0 (`tf.zeros`) or all values set to 1 (`tf.ones`):

```
// 3x5 Tensor with all values set to 0  
const zeros = tf.zeros([3, 5]);  
// Output: [[0, 0, 0, 0, 0],  
//          [0, 0, 0, 0, 0],  
//          [0, 0, 0, 0, 0]]
```

In TensorFlow.js, tensors are immutable; once created, you cannot change their values. Instead you perform operations on them that generate new tensors.

您可以宣告各種維度的張量

- 0 維：`tf.scalar` - 常量
- 1 維：`tf.tensor1d`
- 2 維：`tf.tensor2d`
- 3 維：`tf.tensor3d`
- 4 維：`tf.tensor4d`

張量可以存放常數與變數

- 變數的值可以修改，常數的值不可以修改

Variables

`Variable`s are initialized with a tensor of values. Unlike `Tensor`s, however, their values are mutable. You can assign a new tensor to an existing variable using the `assign` method:

```
const initialValues = tf.zeros([5]);  
const biases = tf.variable(initialValues); // initialize biases  
biases.print(); // output: [0, 0, 0, 0, 0]  
  
const updatedValues = tf.tensor1d([0, 1, 0, 1, 0]);  
biases.assign(updatedValues); // update values of biases  
biases.print(); // output: [0, 1, 0, 1, 0]
```

Variables are primarily used to store and then update values during model training.

而且張量可以進行運算

像是加減乘除與次方等等.....

Available ops include unary ops such as `square`:

```
const d = tf.tensor2d([[1.0, 2.0], [3.0, 4.0]]);
const d_squared = d.square();
d_squared.print();
// Output: [[1, 4 ],
//          [9, 16]]
```

And binary ops such as `add`, `sub`, and `mul`:

```
const e = tf.tensor2d([[1.0, 2.0], [3.0, 4.0]]);
const f = tf.tensor2d([[5.0, 6.0], [7.0, 8.0]]);

const e_plus_f = e.add(f);
e_plus_f.print();
// Output: [[6 , 8 ],
//          [10, 12]]
```

Tensorflow.js 還支援鏈式語法

- 熟悉 jQuery 的朋友們應該會覺得很親切

TensorFlow.js has a chainable API; you can call ops on the result of ops:

```
const sq_sum = e.add(f).square();  
sq_sum.print();  
// Output: [[36 , 64 ],  
//           [100, 144]]  
  
// All operations are also exposed as functions in the main namespace,  
// so you could also do the following:  
const sq_sum = tf.square(tf.add(e, f));
```



也有支援傳統的函數呼叫方式，您不一定要用鏈式語法

您可以使用張量運算組合成函數

```
// Define function
function predict(input) {
  //  $y = a * x^2 + b * x + c$ 
  // More on tf.tidy in the next section
  return tf.tidy(() => {
    const x = tf.scalar(input);

    const ax2 = a.mul(x.square());
    const bx = b.mul(x);
    const y = ax2.add(bx).add(c);

    return y;
  });
}

// Define constants:  $y = 2x^2 + 4x + 8$ 
const a = tf.scalar(2);
const b = tf.scalar(4);
const c = tf.scalar(8);

// Predict output for input of 2
const result = predict(2);
result.print() // Output: 24
```

然後最重要的

- 是用這些張量建立神經網路模型，像是這樣：

You can also use the high-level API `tf.model` to construct a model out of *layers*, which are a popular abstraction in deep learning. The following code constructs a `tf.sequential` model:

```
const model = tf.sequential();
model.add(
  tf.layers.simpleRNN({
    units: 20,
    recurrentInitializer: 'GlorotNormal',
    inputShape: [80, 4]
  })
);

const optimizer = tf.train.sgd(LEARNING_RATE);
model.compile({optimizer, loss: 'categoricalCrossentropy'});
model.fit({x: data, y: labels});
```

There are many different types of layers available in TensorFlow.js. A few examples include `tf.layers.simpleRNN`, `tf.layers.gru`, and `tf.layers.lstm`.

可惜上面這個例子是不完整的，因為沒有定義 `data`, `labels`, `LEARNING_RATE`

目前的 tensorflow.js

- 只支援在瀏覽器裏面執行
- 而不支援直接在 node.js 裡面執行
- 雖然使用的開發工具需要 node.js，但是執行還只能在瀏覽器裏面...

為甚麼會這樣呢？

原因主要在 WebGL 上面

- WebGL 是一個瀏覽器上利用繪圖卡（繪圖處理器 GPU）進行加速的函式庫。
- Tensorflow.js 會利用 WebGL 加速《張量的計算》...

Node. js 裡面沒有 WebGL

- 所以就沒辦法在 node. js 環境裏執行了...

要能充分發揮 Tensorflow.js 的效能

必須好好管理 GPU 的記憶體

Memory Management: dispose and tf.tidy

Because TensorFlow.js uses the GPU to accelerate math operations, it's necessary to manage GPU memory when working with tensors and variables.

TensorFlow.js provide two functions to help with this: `dispose` and `tf.tidy`.

dispose

You can call `dispose` on a tensor or variable to purge it and free up its GPU memory:

```
const x = tf.tensor2d([[0.0, 2.0], [4.0, 6.0]]);
const x_squared = x.square();

x.dispose();
x_squared.dispose();
```

用完的時候就必須盡快釋放 GPU 的記憶體

使用 `tf.tidy()` 函數可以在完成後自動釋放 GPU 上的記憶體

`tf.tidy`

Using `dispose` can be cumbersome when doing a lot of tensor operations. TensorFlow.js provides another function, `tf.tidy`, that plays a similar role to regular scopes in JavaScript, but for GPU-backed tensors.

`tf.tidy` executes a function and purges any intermediate tensors created, freeing up their GPU memory. It does not purge the return value of the inner function.

```
// tf.tidy takes a function to tidy up after
const average = tf.tidy(() => {
  // tf.tidy will clean up all the GPU memory used by tensors inside
  // this function, other than the tensor that is returned.
  //
  // Even in a short sequence of operations like the one below, a number
  // of intermediate tensors get created. So it is a good practice to
  // put your math ops in a tidy!
  const y = tf.tensor1d([1.0, 2.0, 3.0, 4.0]);
  const z = tf.ones([4]);

  return y.sub(z).square().mean();
});

average.print() // Output: 3.5
```

這樣會方便很多

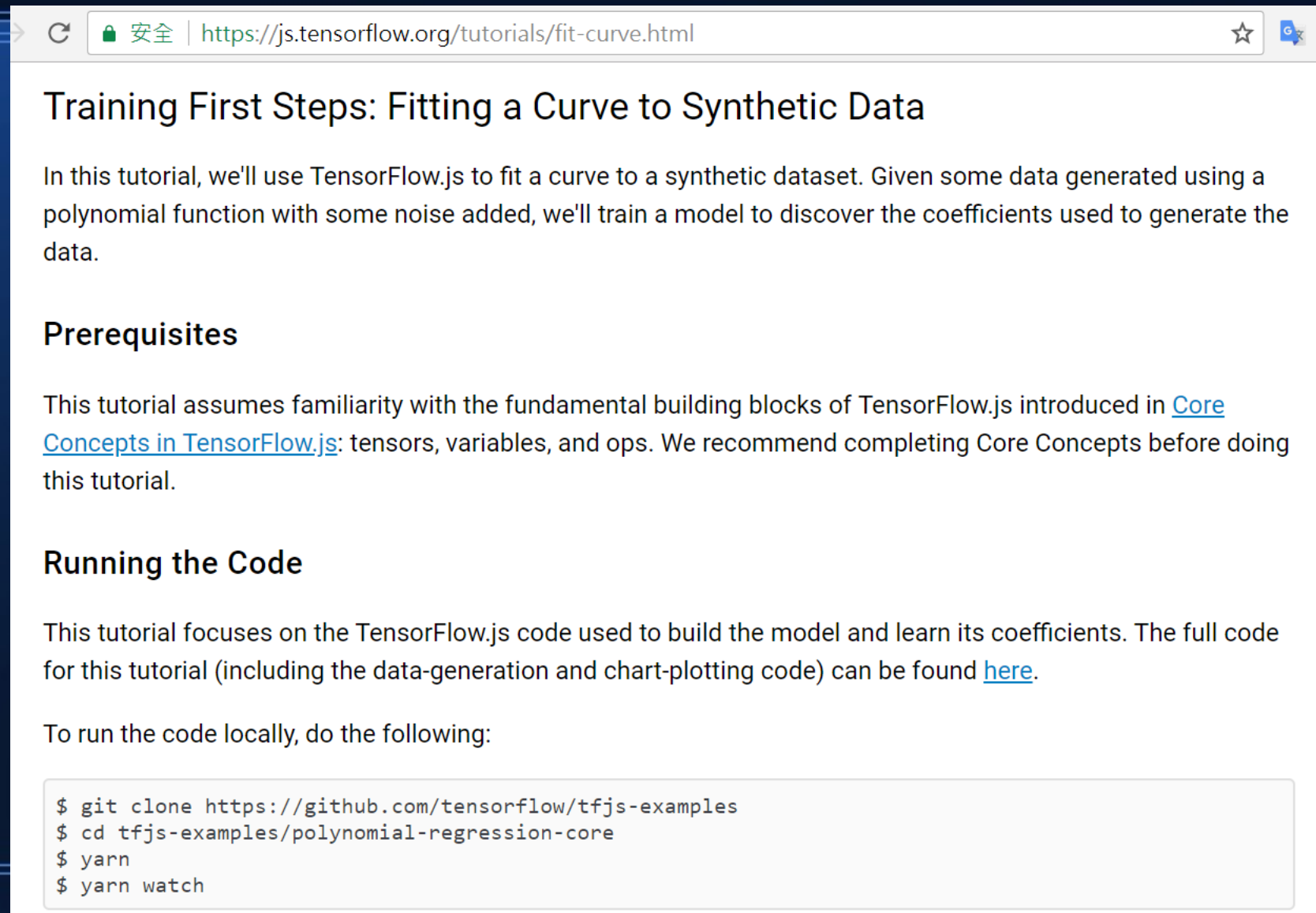
現在

- 您應該已經理解了 tensor+flow 的概念了
- 但或許還不知道深度學習的 tensorflow.js 程式該怎麼寫...

沒關係

- Tensorflow.js 已經為我們準備好了一組 github 範例
- 讓我們直接向範例學習吧！

首先讓我們看看基本的 曲線回歸擬合範例



The screenshot shows a web browser window with the address bar displaying "https://js.tensorflow.org/tutorials/fit-curve.html". The page title is "Training First Steps: Fitting a Curve to Synthetic Data". The main text explains that the tutorial uses TensorFlow.js to fit a curve to a synthetic dataset. It includes sections for prerequisites and running the code, with a terminal snippet at the bottom.

Training First Steps: Fitting a Curve to Synthetic Data

In this tutorial, we'll use TensorFlow.js to fit a curve to a synthetic dataset. Given some data generated using a polynomial function with some noise added, we'll train a model to discover the coefficients used to generate the data.

Prerequisites

This tutorial assumes familiarity with the fundamental building blocks of TensorFlow.js introduced in [Core Concepts in TensorFlow.js](#): tensors, variables, and ops. We recommend completing Core Concepts before doing this tutorial.

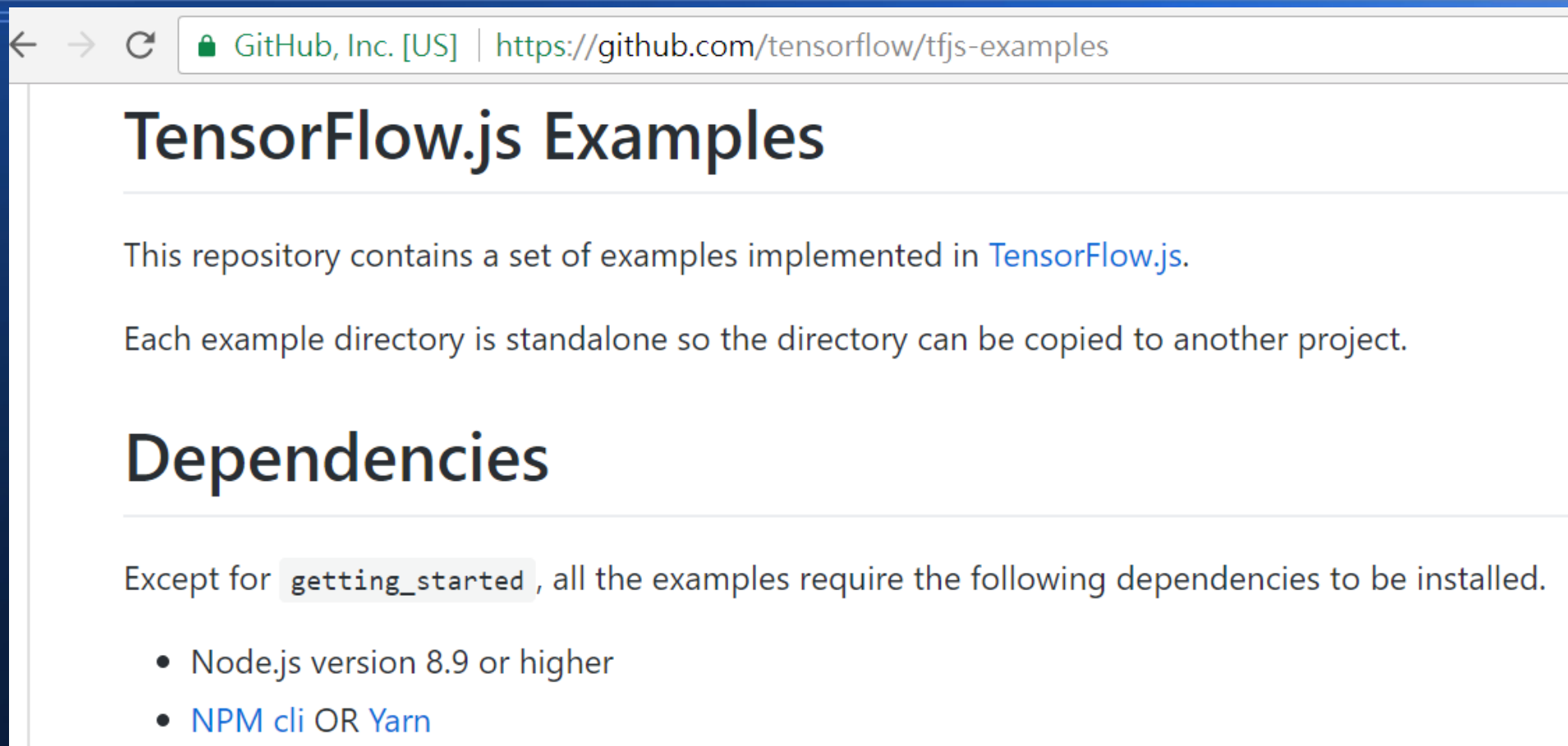
Running the Code

This tutorial focuses on the TensorFlow.js code used to build the model and learn its coefficients. The full code for this tutorial (including the data-generation and chart-plotting code) can be found [here](#).

To run the code locally, do the following:

```
$ git clone https://github.com/tensorflow/tfjs-examples
$ cd tfjs-examples/polynomial-regression-core
$ yarn
$ yarn watch
```

這些範例都在 `tfjs-examples` 專案裡面



The screenshot shows a web browser window with the address bar displaying "GitHub, Inc. [US] | https://github.com/tensorflow/tfjs-examples". The main heading is "TensorFlow.js Examples". Below it, a paragraph states: "This repository contains a set of examples implemented in [TensorFlow.js](#)." Another paragraph follows: "Each example directory is standalone so the directory can be copied to another project." A second heading, "Dependencies", is shown. Below it, a paragraph says: "Except for `getting_started`, all the examples require the following dependencies to be installed." A bulleted list follows with two items: "Node.js version 8.9 or higher" and "[NPM cli](#) OR [Yarn](#)".

← → ↻ [GitHub, Inc. \[US\]](#) | <https://github.com/tensorflow/tfjs-examples>

TensorFlow.js Examples

This repository contains a set of examples implemented in [TensorFlow.js](#).

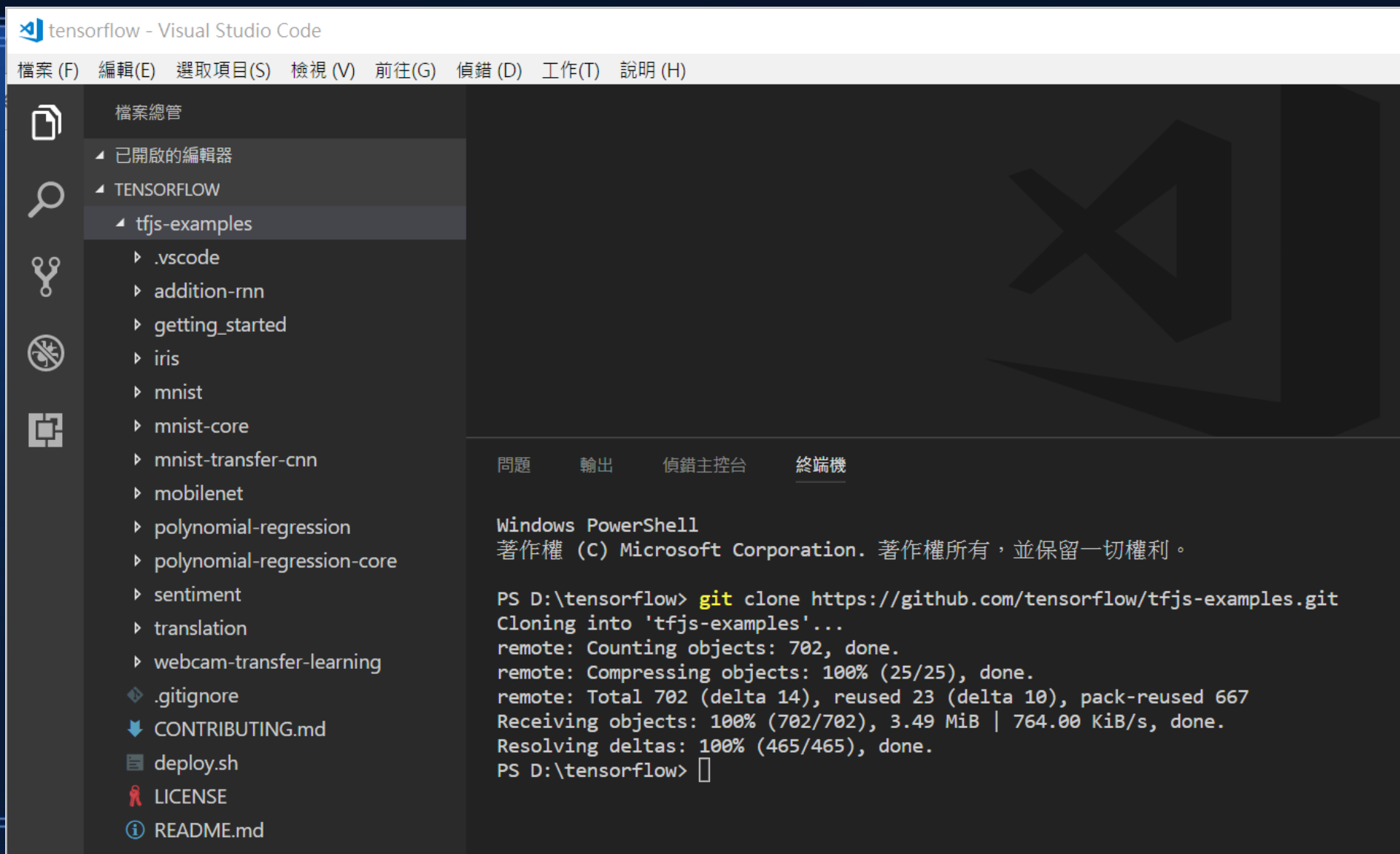
Each example directory is standalone so the directory can be copied to another project.

Dependencies

Except for `getting_started`, all the examples require the following dependencies to be installed.

- Node.js version 8.9 or higher
- [NPM cli](#) OR [Yarn](#)

讓我們先用 git 把它 clone 下來



接著進到 tfjs-examples 中的 polynomial-regression-core 資料夾

Running the Code

This tutorial focuses on the TensorFlow.js code used to build the model and learn its coefficients. The full code for this tutorial (including the data-generation and chart-plotting code) can be found [here](#).

To run the code locally, do the following:

```
$ git clone https://github.com/tensorflow/tfjs-examples
$ cd tfjs-examples/polynomial-regression-core
$ yarn
$ yarn watch
```

The [tfjs-examples/polynomial-regression-core](#) directory above is completely standalone so you can copy it to start your own project.

接著使用 yarn 來安裝模組

然後用 yarn watch 執行專案

Running the Code

This tutorial focuses on the TensorFlow.js code used to build the model and learn its coefficients. The full code for this tutorial (including the data-generation and chart-plotting code) can be found [here](#).

To run the code locally, do the following:

```
$ git clone https://github.com/tensorflow/tfjs-examples
$ cd tfjs-examples/polynomial-regression-core
$ yarn
$ yarn watch
```

The [tfjs-examples/polynomial-regression-core](#) directory above is completely standalone so you can copy it to start your own project.

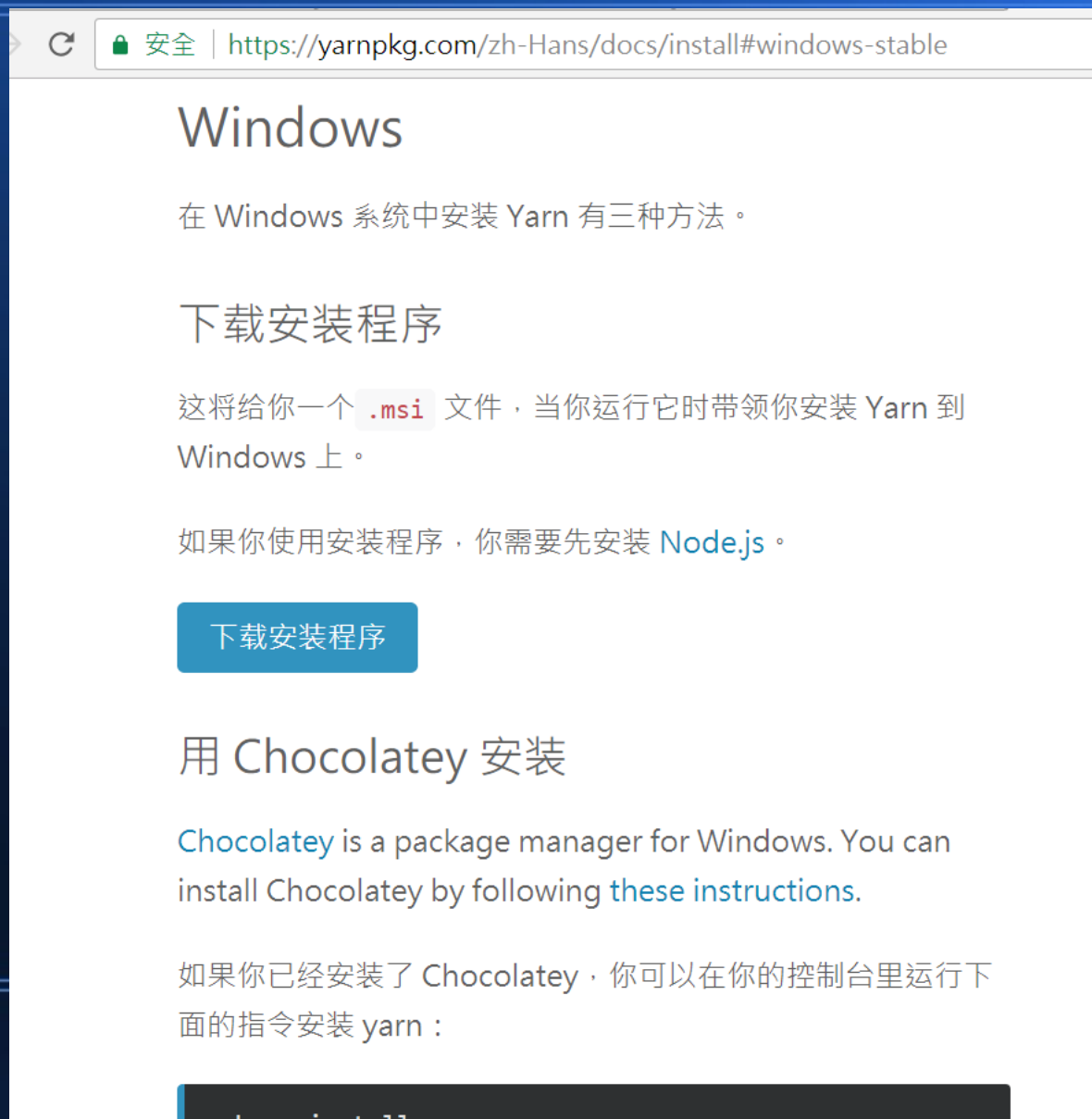
或許你沒聽過 yarn

- yarn 可以說是改良版的 npm

據說速度快很多 ...



您可以先下載安裝 Yarn



然後再回去跑 tensorflow.js 的範例

Running the Code

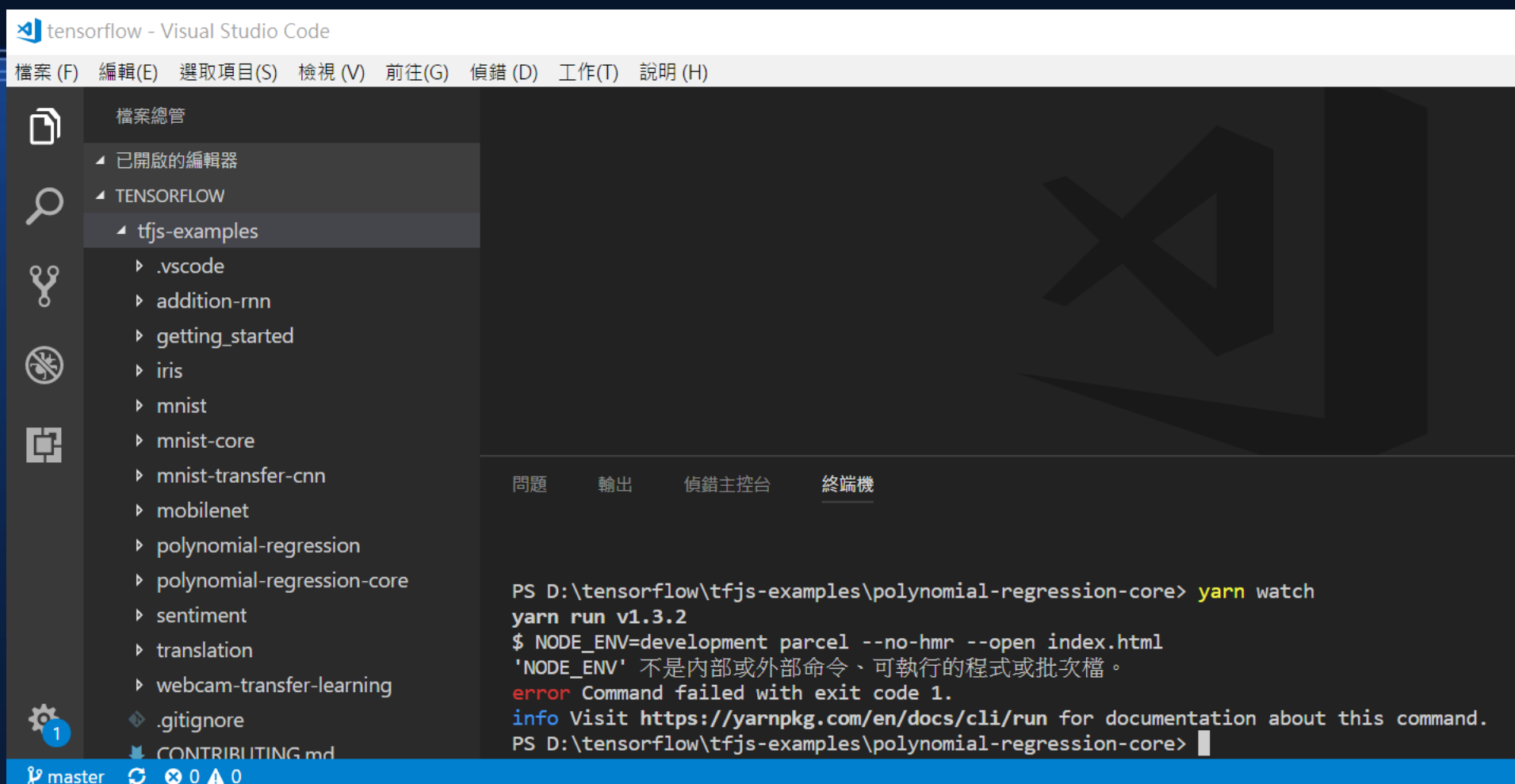
This tutorial focuses on the TensorFlow.js code used to build the model and learn its coefficients. The full code for this tutorial (including the data-generation and chart-plotting code) can be found [here](#).

To run the code locally, do the following:

```
$ git clone https://github.com/tensorflow/tfjs-examples
$ cd tfjs-examples/polynomial-regression-core
$ yarn
$ yarn watch
```

The [tfjs-examples/polynomial-regression-core](#) directory above is completely standalone so you can copy it to start your own project.

悲劇的事情發生了



The screenshot shows the Visual Studio Code interface with the 'tensorflow - Visual Studio Code' window. The left sidebar displays the file explorer with the 'tfjs-examples' folder expanded, showing subfolders like '.vscode', 'addition-rnn', 'getting_started', 'iris', 'mnist', 'mnist-core', 'mnist-transfer-cnn', 'mobilenet', 'polynomial-regression', 'polynomial-regression-core', 'sentiment', 'translation', 'webcam-transfer-learning', and files like '.gitignore' and 'CONTRIBUTING.md'. The bottom panel shows the 'Terminal' tab with the following output:

```
PS D:\tensorflow\tfjs-examples\polynomial-regression-core> yarn watch
yarn run v1.3.2
$ NODE_ENV=development parcel --no-hmr --open index.html
'NODE_ENV' 不是內部或外部命令、可執行的程式或批次檔。
error Command failed with exit code 1.
info Visit https://yarnpkg.com/en/docs/cli/run for documentation about this command.
PS D:\tensorflow\tfjs-examples\polynomial-regression-core>
```

The status bar at the bottom indicates the current branch is 'master' and shows 0 errors and 0 warnings.

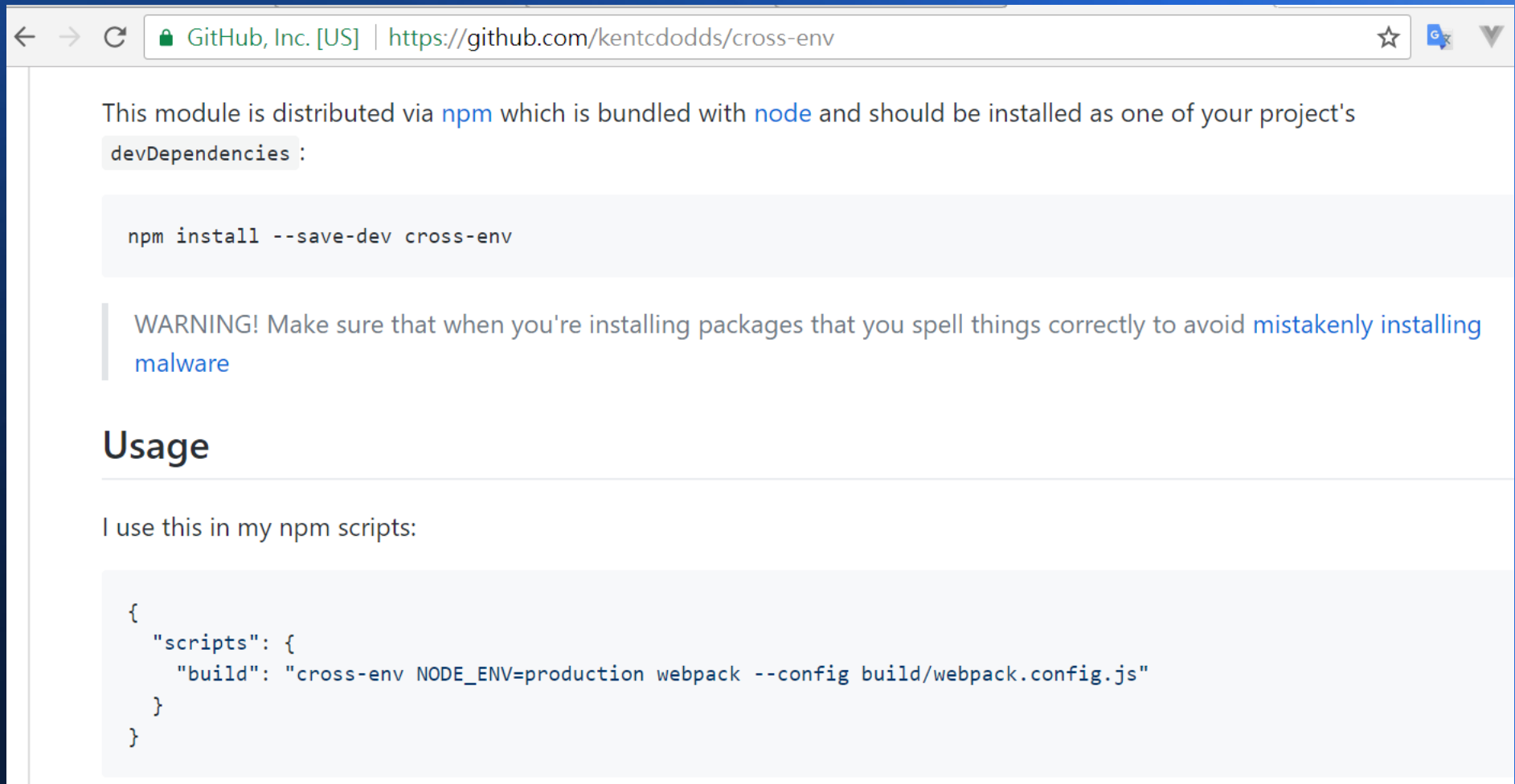
我在 windows 底下執行該範例的 `yarn watch` 指令會失敗了
這是因為 `package.json` 裏使用的是 Linux 風格指令的關係

還好

- 凡事都有解決方案 ...

有一個稱為 `cross-env` 的專案

- 可以幫我們處理這個問題...



The screenshot shows the GitHub repository page for `cross-env` by kentcdodds. The browser address bar shows the URL `https://github.com/kentcdodds/cross-env`. The page content includes instructions on how to install the module via npm, a warning about installing packages correctly, and a section titled "Usage" showing an example of how to use the module in npm scripts.

This module is distributed via `npm` which is bundled with `node` and should be installed as one of your project's `devDependencies`:

```
npm install --save-dev cross-env
```

WARNING! Make sure that when you're installing packages that you spell things correctly to avoid [mistakenly installing malware](#)

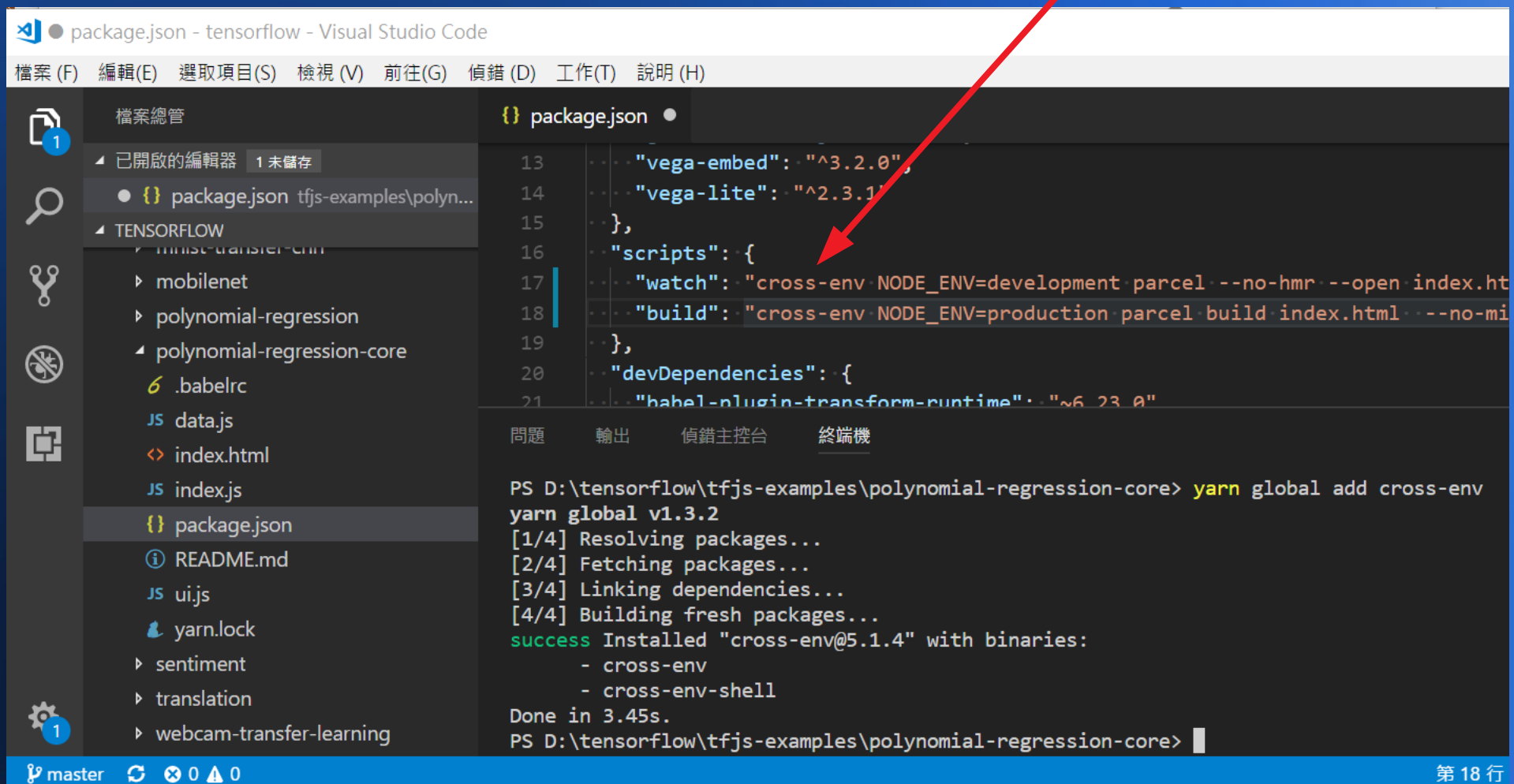
Usage

I use this in my npm scripts:

```
{
  "scripts": {
    "build": "cross-env NODE_ENV=production webpack --config build/webpack.config.js"
  }
}
```


只要先安裝 cross-env

然後在 package.json 的 scripts 裏加入 cross-env



The screenshot shows the Visual Studio Code interface with the `package.json` file open. A red arrow points from the text '然後在 package.json 的 scripts 裏加入 cross-env' to the `scripts` section of the `package.json` file. The `scripts` section contains the following configuration:

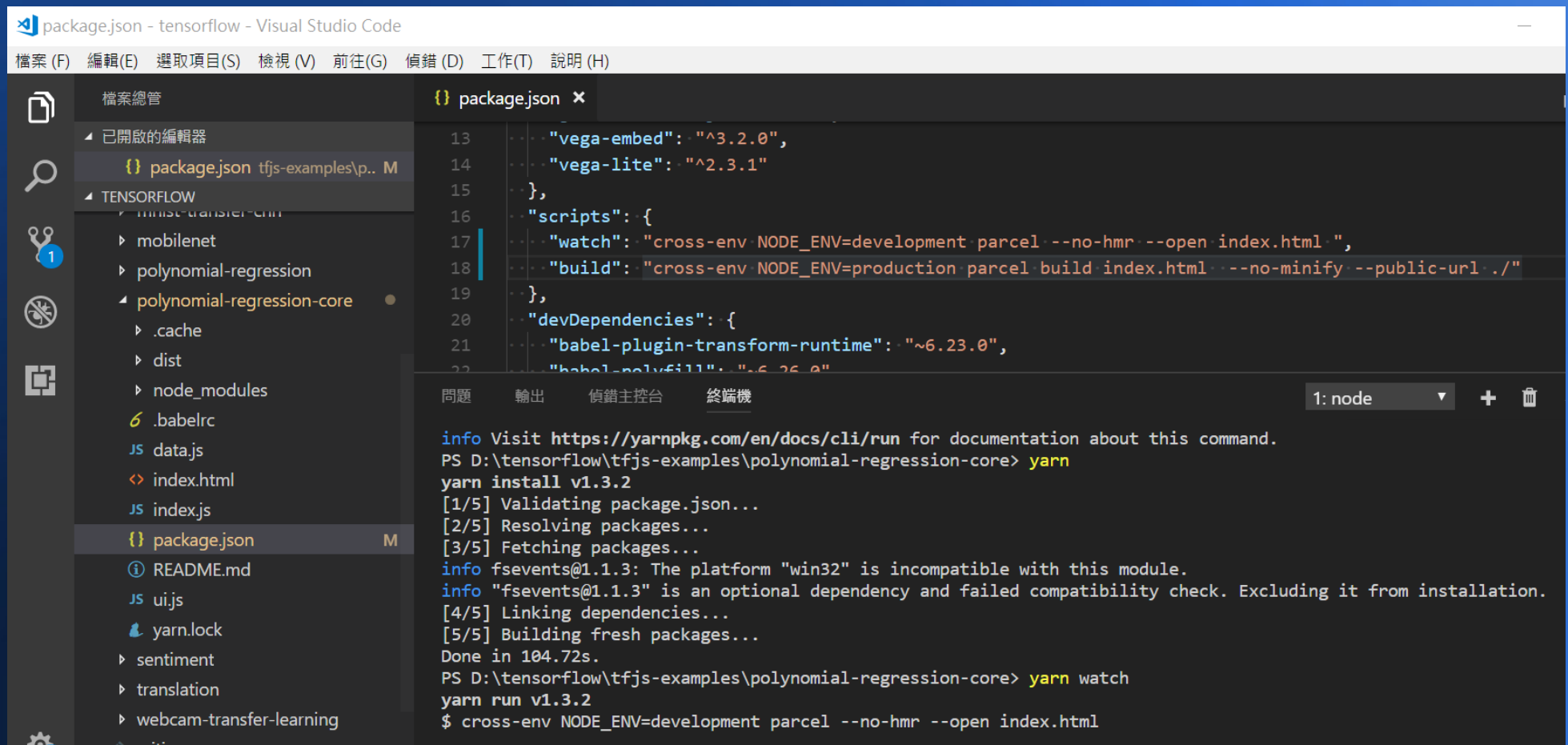
```
"scripts": {
  "watch": "cross-env NODE_ENV=development parcel --no-hmr --open index.html",
  "build": "cross-env NODE_ENV=production parcel build index.html --no-minify"
},
```

Below the code editor, the terminal window shows the command `yarn global add cross-env` being executed, followed by the output:

```
PS D:\tensorflow\tfjs-examples\polynomial-regression-core> yarn global add cross-env
yarn global v1.3.2
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
success Installed "cross-env@5.1.4" with binaries:
  - cross-env
  - cross-env-shell
Done in 3.45s.
PS D:\tensorflow\tfjs-examples\polynomial-regression-core>
```

The status bar at the bottom indicates the current branch is `master` and there are 0 commits, 0 conflicts, and 0 errors.

然後再執行 yarn 安裝該專案使用到的模組
最後用 yarn watch 轉為網頁版就行了



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'tensorflow'. The 'package.json' file is selected and open in the main editor. The file content is as follows:

```
{  
  "name": "tensorflow",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.html",  
  "scripts": {  
    "watch": "cross-env NODE_ENV=development parcel --no-hmr --open index.html",  
    "build": "cross-env NODE_ENV=production parcel build index.html --no-minify --public-url ./",  
  },  
  "devDependencies": {  
    "babel-plugin-transform-runtime": "~6.23.0",  
    "babel-polyfill": "~6.26.0",  
  },  
}
```

Below the editor, the integrated terminal is open, showing the output of the 'yarn' command. The terminal text is:

```
info Visit https://yarnpkg.com/en/docs/cli/run for documentation about this command.  
PS D:\tensorflow\tfjs-examples\polynomial-regression-core> yarn  
yarn install v1.3.2  
[1/5] Validating package.json...  
[2/5] Resolving packages...  
[3/5] Fetching packages...  
info fsevents@1.1.3: The platform "win32" is incompatible with this module.  
info "fsevents@1.1.3" is an optional dependency and failed compatibility check. Excluding it from installation.  
[4/5] Linking dependencies...  
[5/5] Building fresh packages...  
Done in 104.72s.  
PS D:\tensorflow\tfjs-examples\polynomial-regression-core> yarn watch  
yarn run v1.3.2  
$ cross-env NODE_ENV=development parcel --no-hmr --open index.html
```

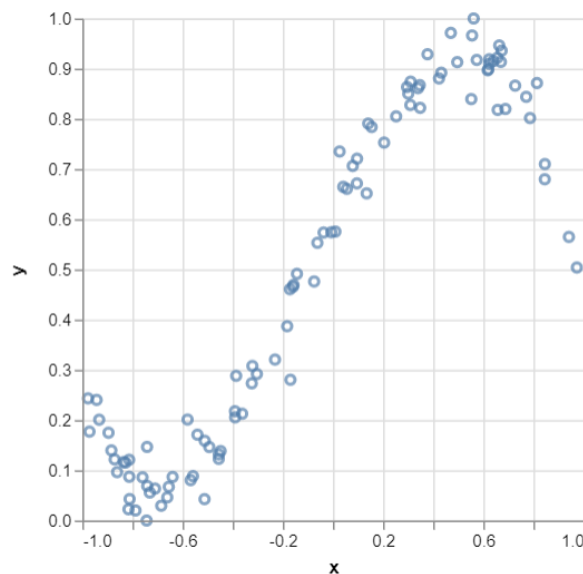
您會看到下列結果

localhost:1234

TensorFlow.js: Fitting a curve to synthetic data

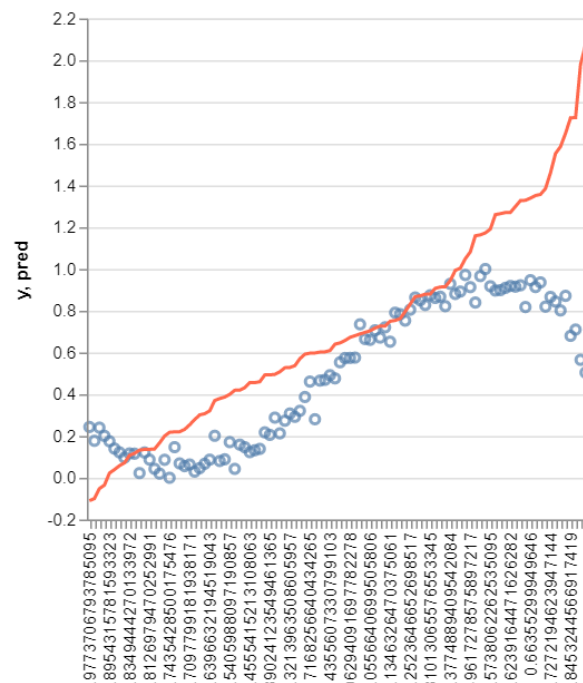
Original Data (Synthetic)

True coefficients: $a=-0.800$, $b=-0.200$, $c=0.900$, $d=0.500$



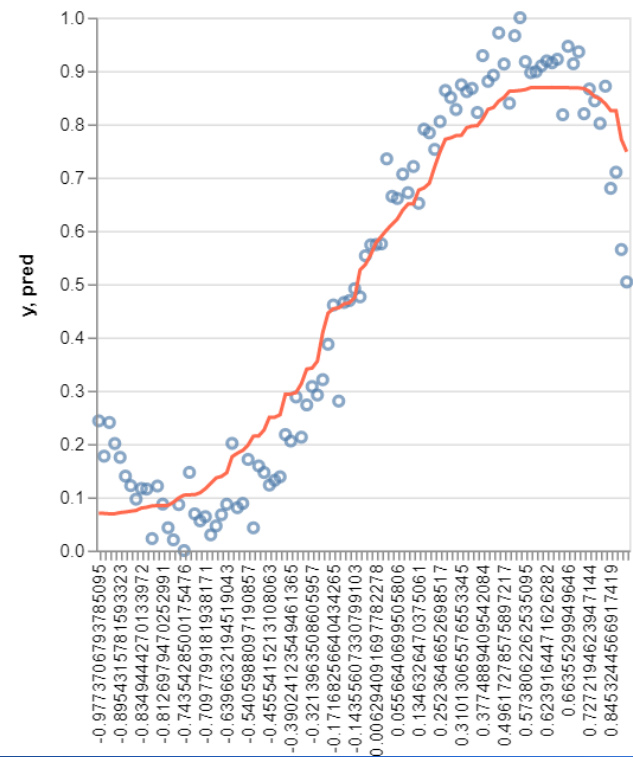
Fit curve with random coefficients (before training)

Random coefficients: $a=0.657$, $b=0.327$, $c=0.495$, $d=0.676$



Fit curve with learned coefficients (after training)

Learned coefficients: $a=-0.402$, $b=-0.183$, $c=0.730$, $d=0.582$



(a) 原始資料

(b) 一開始的亂數擬合結果

(c) 學習完後的擬合結果

效果還不錯

- 問題是、這樣的程式要怎麼寫？

讓我們看看原始碼

<https://github.com/tensorflow/tfjs-examples/blob/master/polynomial-regression-core/index.js>

```
const a = tf.variable(tf.scalar(Math.random()));
const b = tf.variable(tf.scalar(Math.random()));
const c = tf.variable(tf.scalar(Math.random()));
const d = tf.variable(tf.scalar(Math.random()));
```

```
function predict(x) {  
  // y = a * x ^ 3 + b * x ^ 2 + c * x + d  
  return tf.tidy(() => {  
    return a.mul(x.pow(tf.scalar(3, 'int32')))  
      .add(b.mul(x.square()))  
      .add(c.mul(x))  
      .add(d);  
  });  
}
```

設定參數

回歸函數為
 ax^3+bx^2+cx+d

```
function loss(prediction, labels) {  
  // Having a good error function is key for training a  
  const error = prediction.sub(labels).square().mean();  
  return error;  
}
```

損失函數 (能量函數)

```
const numIterations = 75;  
const learningRate = 0.5;  
const optimizer = tf.train.sgd(learningRate);
```

使用隨機梯度下降法

```
async function train(xs, ys, numIterations) {  
  for (let iter = 0; iter < numIterations; iter++) {  
    optimizer.minimize(() => {  
      // Feed the examples into the model  
      const pred = predict(xs);  
      return loss(pred, ys);  
    });  
  
    // Use tf.nextFrame to not block the browser.  
    await tf.nextFrame();  
  }  
}
```

主要訓練程式
最小化損失函數

另外就是產生和繪製資料的程式了

```
export function generateData(numPoints, coeff, sigma = 0.04) {
  return tf.tidy(() => {
    const [a, b, c, d] = [
      tf.scalar(coeff.a), tf.scalar(coeff.b), tf.scalar(coeff.c),
      tf.scalar(coeff.d)
    ];

    const xs = tf.randomUniform([numPoints], -1, 1);

    // Generate polynomial data
    const three = tf.scalar(3, 'int32');
    const ys = a.mul(xs.pow(three))
      .add(b.mul(xs.square()))
      .add(c.mul(xs))
      .add(d)
    // Add random noise to the generated data
    // to make the problem a bit more interesting
    .add(tf.randomNormal([numPoints], 0, sigma));

    // Normalize the y values to the range 0 to 1.
    const ymin = ys.min();
    const ymax = ys.max();
    const yrange = ymax.sub(ymin);
    const ysNormalized = ys.sub(ymin).div(yrange);

    return {
      xs,
      ys: ysNormalized
    };
  });
}
```

```
export async function plotData(container, xs, ys) {
  const xvals = await xs.data();
  const yvals = await ys.data();

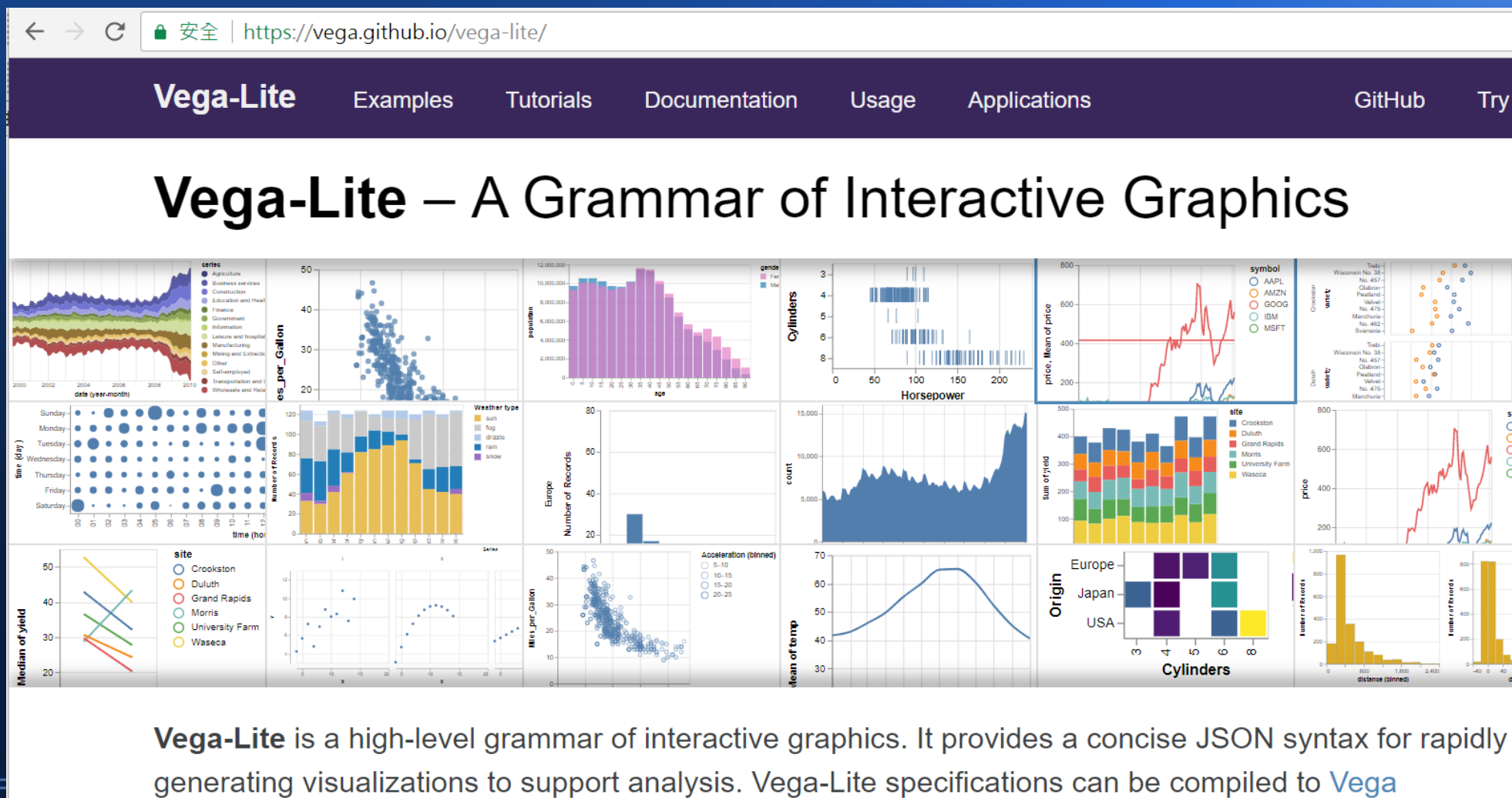
  const values = Array.from(yvals).map((y, i) => {
    return {'x': xvals[i], 'y': yvals[i]};
  });

  const spec = {
    '$schema': 'https://vega.github.io/schema/vega-lite/v2.json',
    'width': 300,
    'height': 300,
    'data': {'values': values},
    'mark': 'point',
    'encoding': {
      'x': {'field': 'x', 'type': 'quantitative'},
      'y': {'field': 'y', 'type': 'quantitative'}
    }
  };

  return renderChart(container, spec, {actions: false});
}
```

這裏使用了 Vega-lite 這個繪圖套件

Vega-Lite 繪圖套件感覺還蠻厲害的



不過重點還是機器學習

<https://github.com/tensorflow/tfjs-examples/blob/master/polynomial-regression-core/index.js>

```
const a = tf.variable(tf.scalar(Math.random()));
const b = tf.variable(tf.scalar(Math.random()));
const c = tf.variable(tf.scalar(Math.random()));
const d = tf.variable(tf.scalar(Math.random()));
```

設定參數

```
function predict(x) {
  // y = a * x ^ 3 + b * x ^ 2 + c * x + d
  return tf.tidy(() => {
    return a.mul(x.pow(tf.scalar(3, 'int32')))
      .add(b.mul(x.square()))
      .add(c.mul(x))
      .add(d);
  });
}
```

回歸函數為

ax^3+bx^2+cx+d

```
function loss(prediction, labels) {
  // Having a good error function is key for training a
  const error = prediction.sub(labels).square().mean();
  return error;
}
```

損失函數 (能量函數)

```
const numIterations = 75;
const learningRate = 0.5;
const optimizer = tf.train.sgd(learningRate);
```

使用隨機梯度下降法

```
async function train(xs, ys, numIterations) {
  for (let iter = 0; iter < numIterations; iter++) {
    optimizer.minimize(() => {
      // Feed the examples into the model
      const pred = predict(xs);
      return loss(pred, ys);
    });
  }
}
```

主要訓練程式

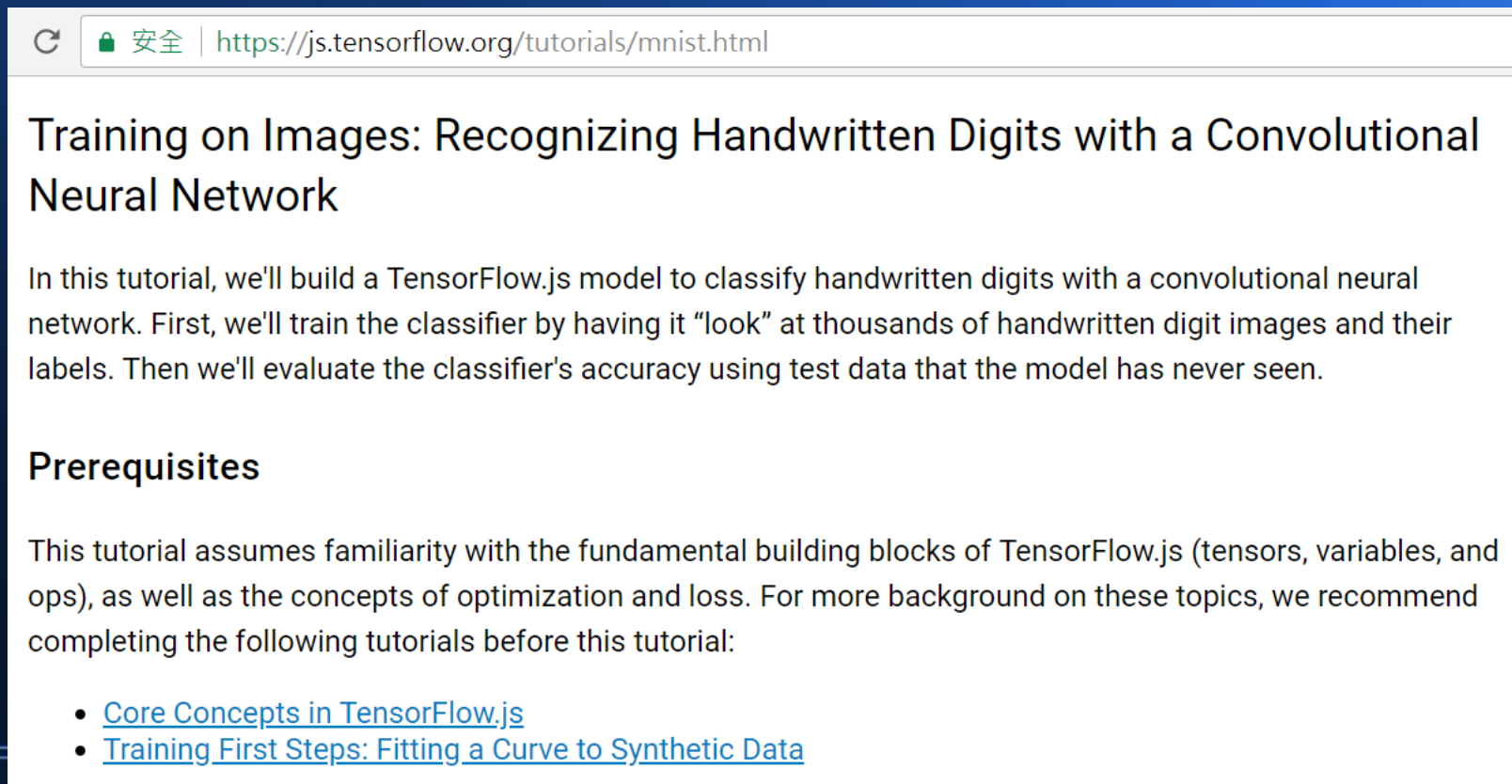
最小化損失函數

但是、上面的曲線擬合範例

- 只是機器學習的入門範例
- 並非深度學習的重點

深度學習的重頭戲

- 是用卷積神經網路 CNN 辨識影像



The screenshot shows a web browser window with the address bar displaying "https://js.tensorflow.org/tutorials/mnist.html". The page title is "Training on Images: Recognizing Handwritten Digits with a Convolutional Neural Network". The main text describes the tutorial's goal: building a TensorFlow.js model to classify handwritten digits using a convolutional neural network. It mentions training on thousands of handwritten digit images and evaluating accuracy on test data. A "Prerequisites" section follows, stating that familiarity with TensorFlow.js fundamentals (tensors, variables, ops) and optimization/loss concepts is required. It recommends completing two other tutorials: "Core Concepts in TensorFlow.js" and "Training First Steps: Fitting a Curve to Synthetic Data".

安全 | <https://js.tensorflow.org/tutorials/mnist.html>

Training on Images: Recognizing Handwritten Digits with a Convolutional Neural Network

In this tutorial, we'll build a TensorFlow.js model to classify handwritten digits with a convolutional neural network. First, we'll train the classifier by having it “look” at thousands of handwritten digit images and their labels. Then we'll evaluate the classifier's accuracy using test data that the model has never seen.

Prerequisites

This tutorial assumes familiarity with the fundamental building blocks of TensorFlow.js (tensors, variables, and ops), as well as the concepts of optimization and loss. For more background on these topics, we recommend completing the following tutorials before this tutorial:

- [Core Concepts in TensorFlow.js](#)
- [Training First Steps: Fitting a Curve to Synthetic Data](#)

而 MNIST 手寫數字辨識資料庫

- 可以說是影像辨識領域的果蠅
- 所有影像辨識演算法，都會先用 MNIST 進行測試 ...

Tensorflow.js 裏有個 MNIST 的範例

- 可惜我的 windows 建置時會失敗

```
PS D:\tensorflow\tfjs-examples\mnist> yarn
yarn install v1.3.2
[1/5] Validating package.json...
[2/5] Resolving packages...
[3/5] Fetching packages...
info fsevents@1.1.3: The platform "win32" is incompatible with this module.
info "fsevents@1.1.3" is an optional dependency and failed compatibility check. Excluding it from installation.
[4/5] Linking dependencies...
[5/5] Building fresh packages...
[1/3] . canvas: d:\tensorflow\tfjs-examples\mnist\node_modules\canvas\src\Canvas.h(19): fatal error C1083: Canno
[-/3] . waiting...
[-/3] . waiting...
[-/3] . waiting...
warning Error running install script for optional dependency: "D:\\tensorflow\\tfjs-examples\\mnist\\node_modules\\canvas
: Command failed.\nExit code: 1\nCommand: node-gyp rebuild\nArguments: \nDirectory: D:\\tensorflow\\tfjs-examples\\mnist\\
```

這看來像是 node-gyp 沒裝的問題

- 所以我就裝了 node-gyp

```
PS D:\tensorflow> yarn global add node-gyp
yarn global v1.3.2
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
success Installed "node-gyp@3.6.2" with binaries:
  - node-gyp
Done in 16.57s.
```

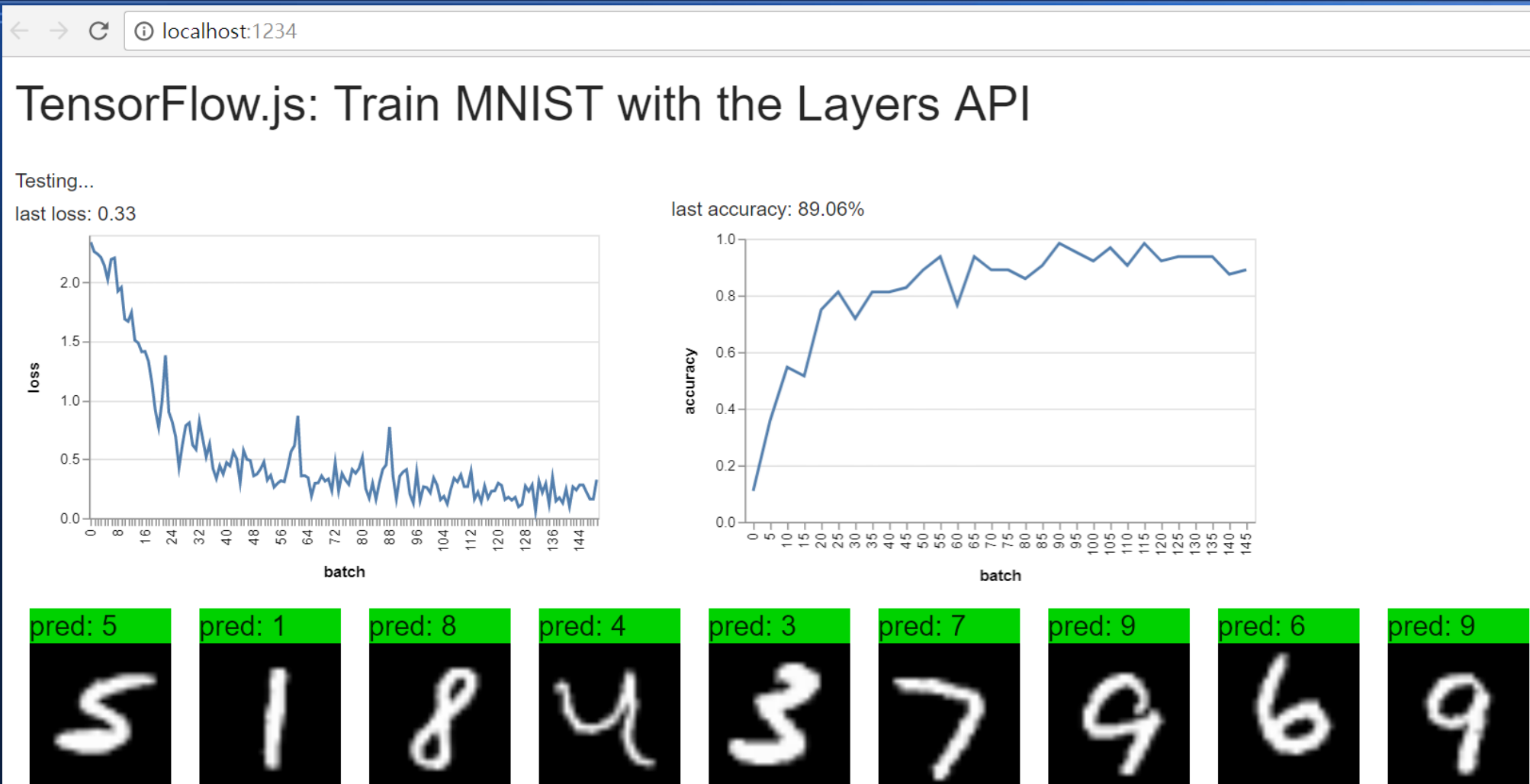
然後就可以用了

- 不管是 build 還是 watch 都沒問題

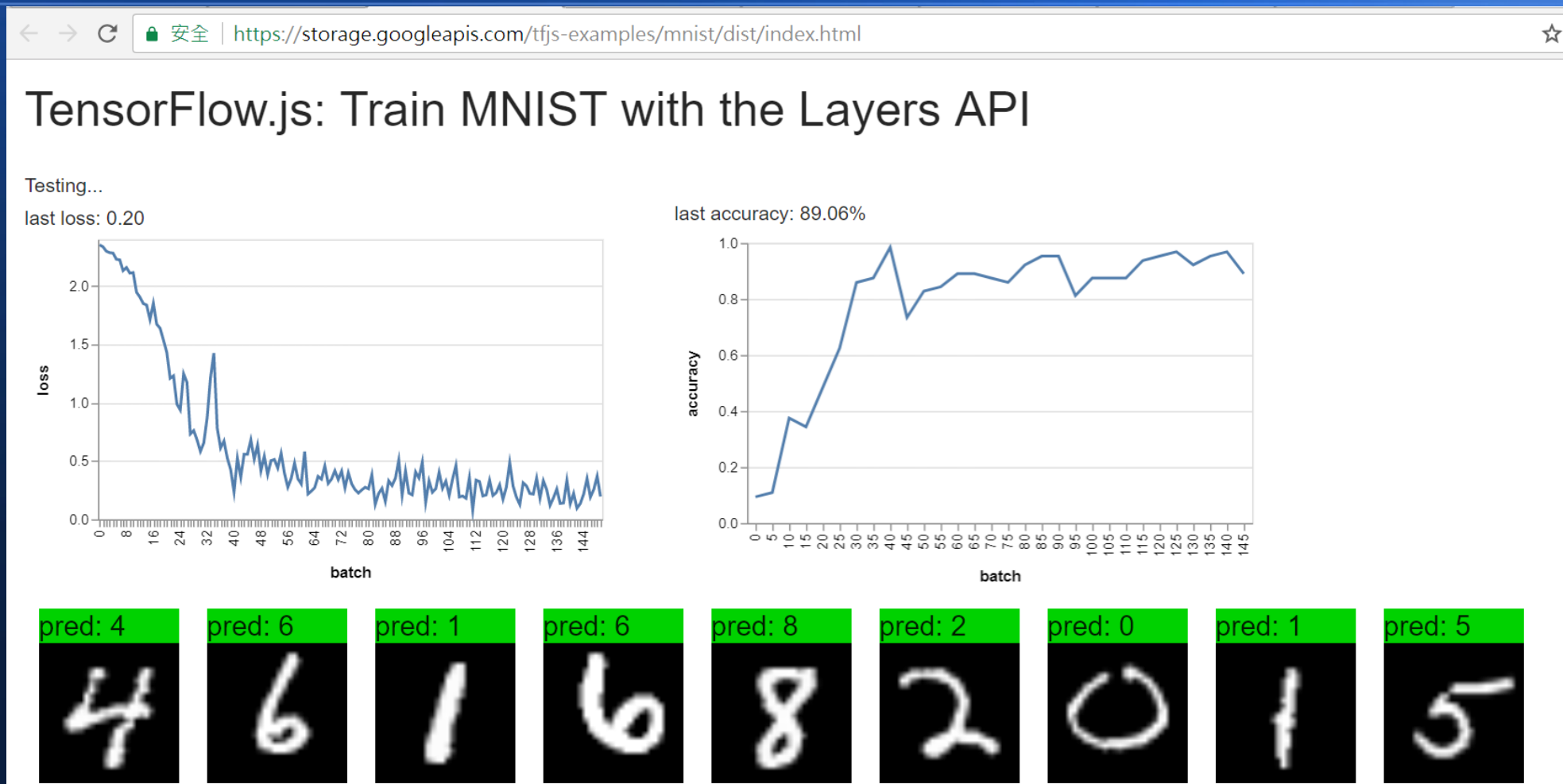
```
PS D:\tensorflow\tfjs-examples\mnist> yarn build
yarn run v1.3.2
$ cross-env NODE_ENV=production parcel build index.html --no-minify --public-url ./
✓ Built in 76.48s.

dist\tfjs-examples-mnist.js    !!  4.39 MB    4902.68s
dist\index.html                2.07 KB     2.77s
Done in 80.37s.
PS D:\tensorflow\tfjs-examples\mnist> yarn watch
yarn run v1.3.2
$ cross-env NODE_ENV=development parcel --no-hmr --open index.html
Server running at http://localhost:1234
✓ Built in 83.38s.
```

訓練辨識的結果像這樣



如果您不想重新建置範例 也有線上展示版可以看



<https://storage.googleapis.com/tfjs-examples/mnist/dist/index.html>

統計結果正確率是 89%



由於只訓練了短短幾秒鐘

- 就能到達 89% ，應該還不算是太差

該 CNN 主要的神經網路模型如下

```
const model = tf.sequential();

model.add(tf.layers.conv2d({
  inputShape: [28, 28, 1],
  kernelSize: 5,
  filters: 8,
  strides: 1,
  activation: 'relu',
  kernelInitializer: 'varianceScaling'
}));

model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

model.add(tf.layers.conv2d({
  kernelSize: 5,
  filters: 16,
  strides: 1,
  activation: 'relu',
  kernelInitializer: 'varianceScaling'
}));

model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

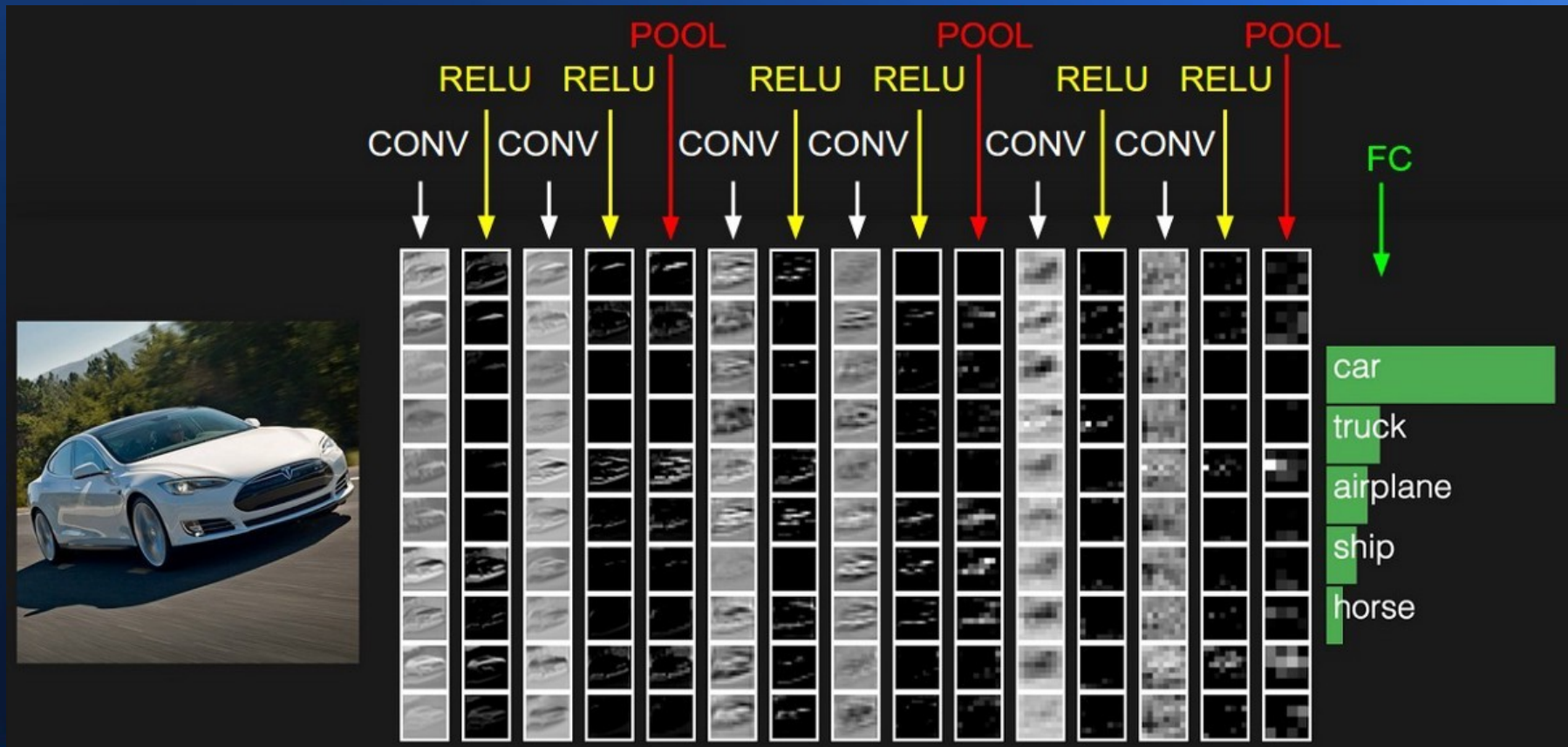
model.add(tf.layers.flatten());

model.add(tf.layers.dense(
  {units: 10, kernelInitializer: 'varianceScaling', activation: 'softmax'}));
```

這個模型就是經典的 CNN 卷積網路

卷積 Conv+ 整流 ReLU+ 池化 Pool

最後再加上一層 《全連接網路 FC》



再仔細看一遍

```
const model = tf.sequential();

model.add(tf.layers.conv2d({
  inputShape: [28, 28, 1],
  kernelSize: 5,
  filters: 8,
  strides: 1,
  activation: 'relu',
  kernelInitializer: 'varianceScaling'
}));

model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

model.add(tf.layers.conv2d({
  kernelSize: 5,
  filters: 16,
  strides: 1,
  activation: 'relu',
  kernelInitializer: 'varianceScaling'
}));

model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

model.add(tf.layers.flatten());

model.add(tf.layers.dense({
  units: 10, kernelInitializer: 'varianceScaling', activation: 'softmax'
}));
```

捲積層 Conv 1

輸出前使用 ReLU

池化層 Pool 1

捲積層 Conv 2

輸出前使用 ReLU

池化層 Pool 2

把原本多維的張量整平成一維

最後的全連接層

最後、一樣是用隨機梯度下降法 **sgd** 學習

```
const LEARNING_RATE = 0.15;
const optimizer = tf.train.sgd(LEARNING_RATE);
model.compile({
  optimizer: optimizer,
  loss: 'categoricalCrossentropy',
  metrics: ['accuracy'],
});
```

剩下的程式碼

主要就是處理

1. 載入影像

2. 批次測試

3. 呈現結果

那些瑣事了

```
for (let i = 0; i < TRAIN_BATCHES; i++) {
  const batch = data.nextTrainBatch(BATCH_SIZE);

  let testBatch;
  let validationData;
  // Every few batches test the accuracy of the mode.
  if (i % TEST_ITERATION_FREQUENCY === 0) {
    testBatch = data.nextTestBatch(TEST_BATCH_SIZE);
    validationData = [
      testBatch.xs.reshape([TEST_BATCH_SIZE, 28, 28, 1]), testBatch.labels
    ];
  }

  // The entire dataset doesn't fit into memory so we call fit repeatedly
  // with batches.
  const history = await model.fit(
    batch.xs.reshape([BATCH_SIZE, 28, 28, 1]),
    batch.labels,
    {
      batchSize: BATCH_SIZE,
      validationData,
      epochs: 1
    });

  const loss = history.history.loss[0];
  const accuracy = history.history.acc[0];

  // ... plotting code ...
}
```

看完卷積神經網路 CNN 的範例之後

- 您應該會感覺到 tensorflow.js 的程式
其實不難寫

目前 tensorflow.js 包含了下列網路

- 基礎的多層全連接感知器

MLP (dense)

Basic

tf.layers.activation
tf.layers.dense
tf.layers.dropout
tf.layers.embedding
tf.layers.flatten
tf.layers.repeatVector

Advanced Activation

tf.layers.elu
tf.layers.leakyReLU
tf.layers.softmax
tf.layers.thresholdedReLU

- 還有卷積神經網路

CNN (Conv+Pool)

Convolutional

tf.layers.conv1d
tf.layers.conv2d
tf.layers.conv2dTranspose
tf.layers.depthwiseConv2d
tf.layers.separableConv2d

Pooling

tf.layers.averagePooling1d
tf.layers.averagePooling2d
tf.layers.globalAveragePooling1d
tf.layers.globalAveragePooling2d
tf.layers.globalMaxPooling1d
tf.layers.globalMaxPooling2d
tf.layers.maxPooling1d
tf.layers.maxPooling2d

另外還包含了循環神經網路 RNN 與 LSTM

Recurrent

tf.layers.gru
tf.layers.gruCell
tf.layers.lstm
tf.layers.lstmCell
tf.layers.rnn
tf.layers.simpleRNN
tf.layers.simpleRNNCell
tf.layers.stackedRNNCells

Wrapper

tf.layers.bidirectional
tf.layers.timeDistributed

Classes

tf.layers.Layer
tf.RNNCell

Inputs

tf.layers.inputLayer

Padding

tf.layers.zeroPadding2d

其 RNN 類型的 GRU 網路建模方式如下

```
function createAndCompileModel(layers, hiddenSize, rnnType='GRU', digits, vocabularySize) {  
  const maxlen = digits + 1 + digits;  
  const model = tf.sequential();  
  model.add(tf.layers.gru({  
    units: hiddenSize,  
    recurrentInitializer: 'glorotNormal',  
    inputShape: [maxlen, vocabularySize]  
  }));  
  model.add(tf.layers.repeatVector({n: digits + 1}));  
  model.add(tf.layers.gru({  
    units: hiddenSize,  
    recurrentInitializer: 'glorotNormal',  
    returnSequences: true  
  }));  
  model.add(tf.layers.timeDistributed(  
    {layer: tf.layers.dense({units: vocabularySize})});  
  model.add(tf.layers.activation({activation: 'softmax'}));  
  model.compile({  
    loss: 'categoricalCrossentropy',  
    optimizer: 'adam',  
    metrics: ['accuracy']  
  });  
  return model;  
}
```

上述這段程式是從 addition-rnn 範例裏抽出來簡化的

該範例是用循環神經網路 學習兩個數字相加的

A screenshot of a web browser window. The address bar shows '127.0.0.1:8080'. Below the address bar, there are three links: 'Export as PNG', 'View Source', and 'Open in Vega Editor'. A list of 20 addition problems is displayed. Most are in green, but four are in red: '4+47 = 50', '19+80 = 98', and '80+17 = 98'.

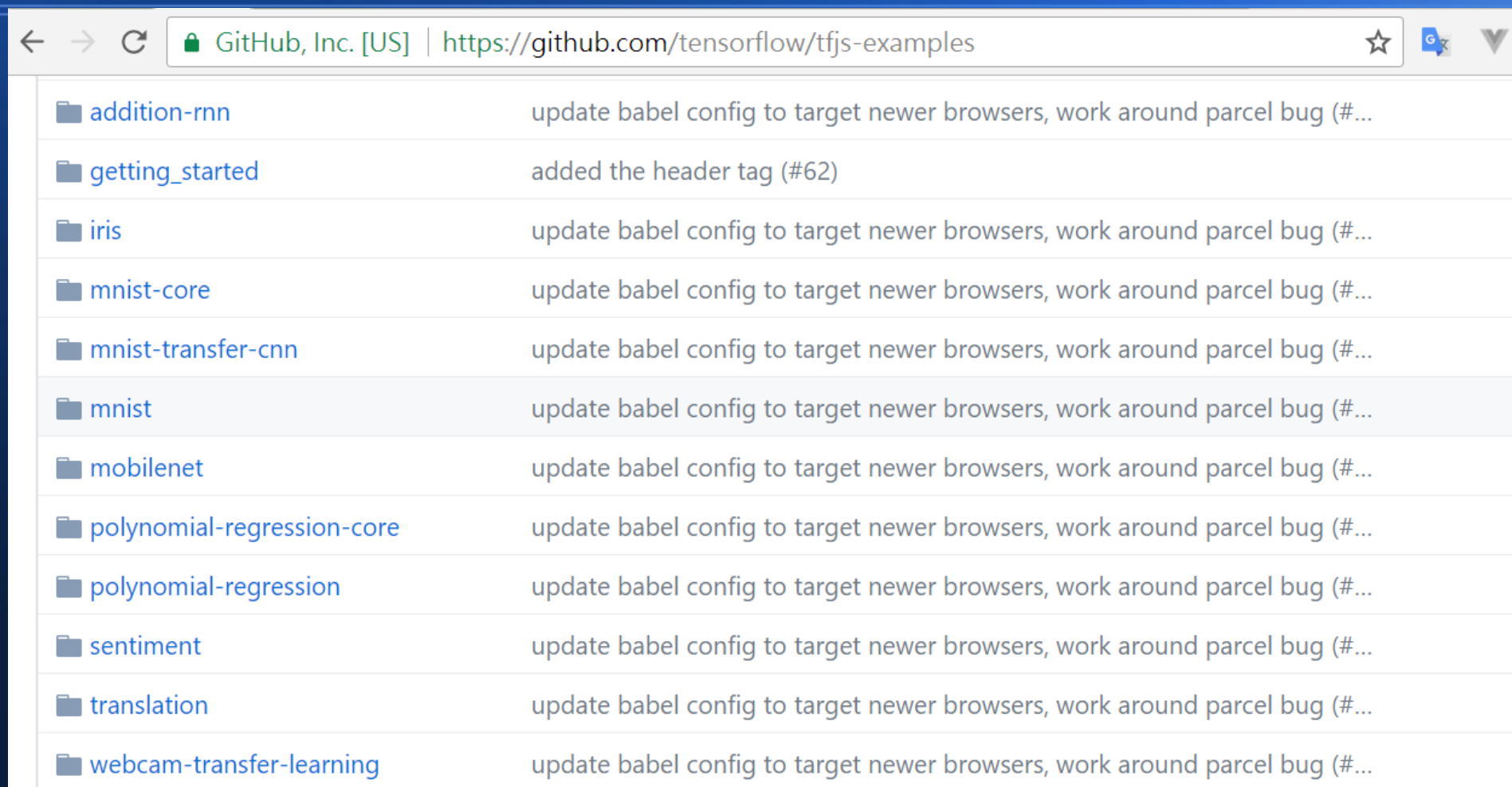
```
< > ↻ ⓘ 127.0.0.1:8080  
Export as PNG View Source Open in Vega Editor  
57+65 = 122  
41+3 = 44  
45+26 = 71  
4+47 = 50  
35+84 = 119  
87+94 = 181  
24+42 = 66  
53+34 = 87  
88+9 = 97  
65+39 = 104  
62+43 = 105  
48+41 = 89  
98+31 = 129  
19+80 = 98  
38+86 = 124  
58+33 = 91  
80+17 = 98
```

線上版網址如下

<https://storage.googleapis.com/tfjs-examples/addition-rnn/dist/index.html>

您可以試試看！

官方的 tfjs-examples 裏還有一些進階範例



addition-rnn	update babel config to target newer browsers, work around parcel bug (#...
getting_started	added the header tag (#62)
iris	update babel config to target newer browsers, work around parcel bug (#...
mnist-core	update babel config to target newer browsers, work around parcel bug (#...
mnist-transfer-cnn	update babel config to target newer browsers, work around parcel bug (#...
mnist	update babel config to target newer browsers, work around parcel bug (#...
mobilenet	update babel config to target newer browsers, work around parcel bug (#...
polynomial-regression-core	update babel config to target newer browsers, work around parcel bug (#...
polynomial-regression	update babel config to target newer browsers, work around parcel bug (#...
sentiment	update babel config to target newer browsers, work around parcel bug (#...
translation	update babel config to target newer browsers, work around parcel bug (#...
webcam-transfer-learning	update babel config to target newer browsers, work around parcel bug (#...

有不少 RNN,LSTM 的範例，像是 sentiment, translation, addition-rnn 等 ...

像是情感分析 sentiment

- 還有英語對法語的翻譯系統 Translation

另外官網裏還有一些

專為手機打造的 tensorflow.js 展示程式

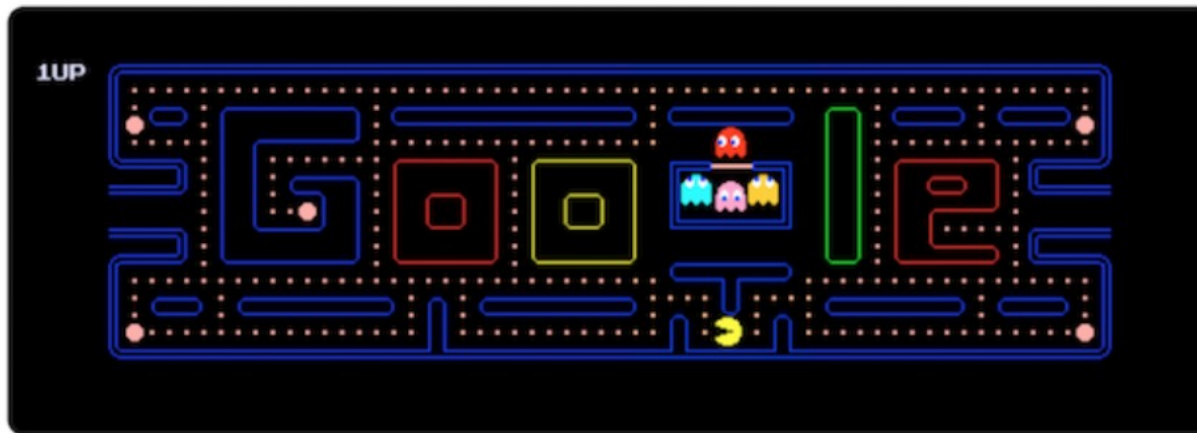
像是立即找出題目指定的物件以供 手機辨識的遊戲 EMOJI SCAVENGER HUNT

EMOJI SCAVENGER HUNT

Use your phone's camera to identify emojis in the real world. Can you find all the emojis before time expires?

[GO TO DEMO!](#)[CODE](#)

自動打小精靈的應用



1

Learning rate

0.0001

Batch size

0.4

TRAIN

PLAY

WEBCAM CONTROLLER

Play Pac-Man using images trained in your browser.

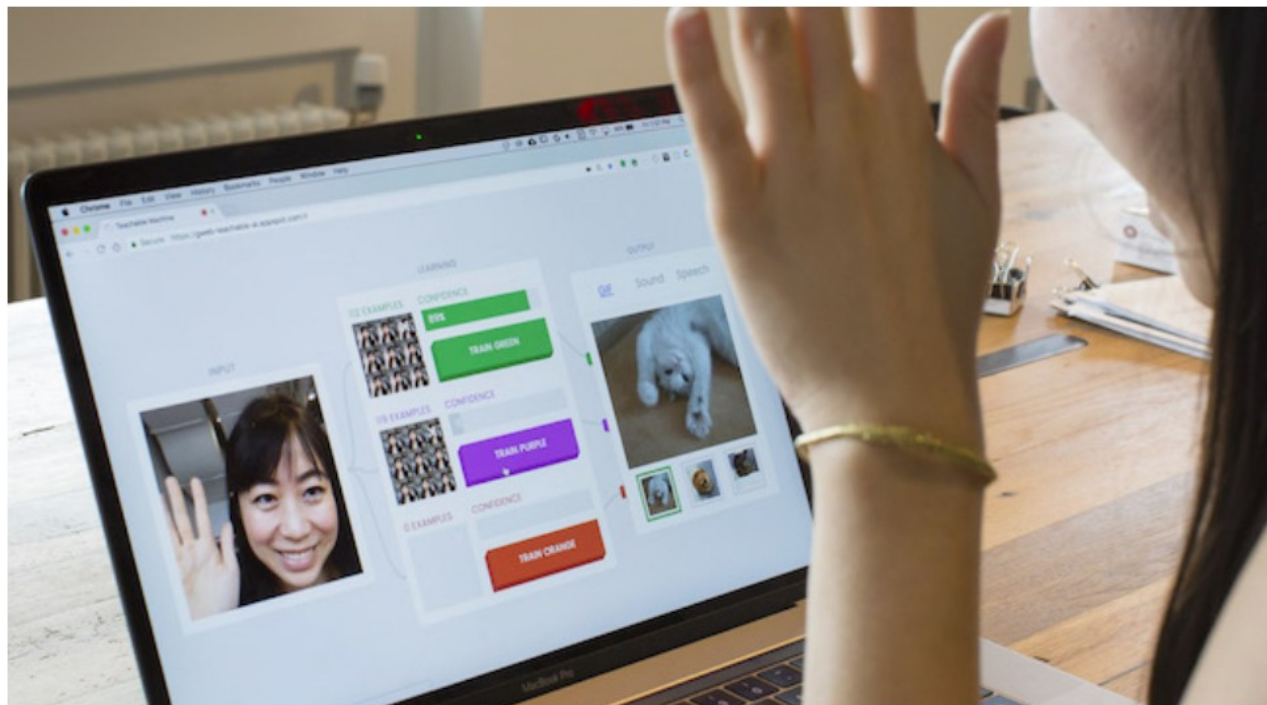
[GO TO DEMO!](#)

[CODE](#)

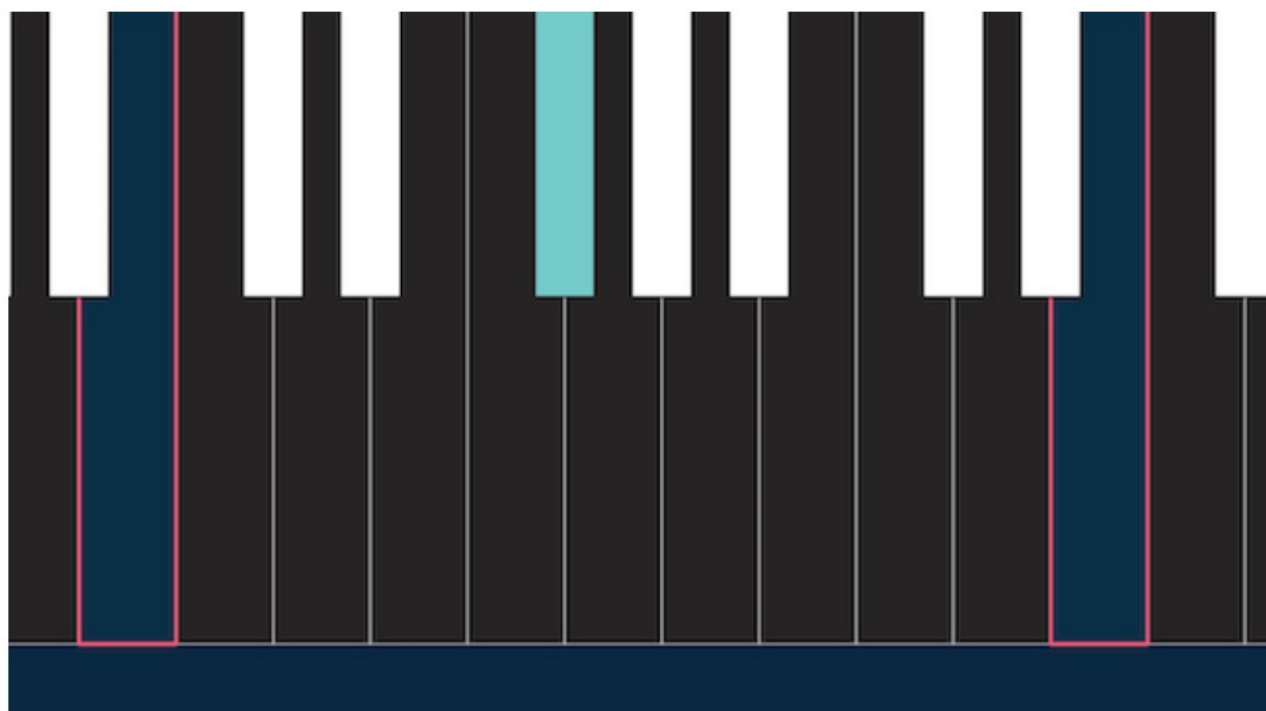
教你神經網路原理的程式

TEACHABLE MACHINE

No coding required! Teach a machine to recognize images and play sounds.

[GO TO DEMO!](#)[CODE](#)

用 RNN 彈鋼琴的範例



PERFORMANCE RNN

Enjoy a real-time piano performance by a neural network

[GO TO DEMO!](#)[CODE](#)

資源也算相當完整了

雖然還沒辦法和 Python 抗衡

- 但已經是我看過的 JavaScript 深度學習套件中最完整的一個...

該有的方法也都有了

Gradients

tf.grad
tf.grads
tf.customGrad
tf.valueAndGrad
tf.valueAndGrads
tf.variableGrads

Optimizers

tf.train.sgd
tf.train.momentum
tf.train.adagrad
tf.train.adadelta
tf.train.adam
tf.train.adamax
tf.train.rmsprop

tf.ceil
tf.clipByValue
tf.cos
tf.cosh
tf.elu
tf.exp
tf.expm1
tf.floor
tf.leakyRelu
tf.log
tf.log1p
tf.neg
tf.prelu
tf.reciprocal
tf.relu
tf.round
tf.rsqrt
tf.selu
tf.sigmoid

Reduction

tf.argmax
tf.argMin
tf.logSumExp
tf.max
tf.mean
tf.min
tf.sum

Normalization

tf.batchNormalization
tf.localResponseNorm
tf.moments
tf.softmax

OPERATIONS

Arithmetic

tf.add
tf.sub
tf.mul
tf.div
tf.maximum
tf.minimum
tf.mod
tf.pow
tf.squaredDifference

Basic math

tf.abs
tf.acos
tf.acosh

除了無法在 node.js 裏直接執行

- 還有必須透過 WebGL 而不是直接呼叫 CUDA 函式庫，效能可能會比 Python 差上不少之外

目前應該是 JavaScript 程式人

- 在深度學習領域的第一選擇了

而 tensorflow.js 的競爭對手之一
是原本 node.js 的創始人 Ryan Dahl 所創建的 propelml



The screenshot shows a web browser displaying a news article on the iThome website. The URL in the address bar is <https://www.ithome.com.tw/news/121715>. The article title is "Node.js創辦人光環加持，科學運算用 Javascript函式庫Propel登場". The text describes how Ryan Dahl, the creator of Node.js, has created Propel, a JavaScript library for scientific computing. It mentions that Propel is a GPU-backed numpy-like infrastructure. Below the text, there is a code snippet for using Propel in a notebook, showing imports and a function definition with nested gradient calculations.

← → ↻ 安全 | <https://www.ithome.com.tw/news/121715> ☆ 通知 菜單

iThome

Node.js創辦人光環加持，科學運算用 Javascript函式庫Propel登場

Propel的創始人，原為Node.js的創辦人Ryan Dahl認為，JavaScript是一種快速且動態的語言，很適合用於各種科學目的的計算。

文/ 李建興 | 2018-03-09 發表

讚 4.8 萬 按讚加入iThome粉絲團 讚 429 分享 G+

● Propel

NOTEBOOK SIGN IN

Differential Programming in JavaScript

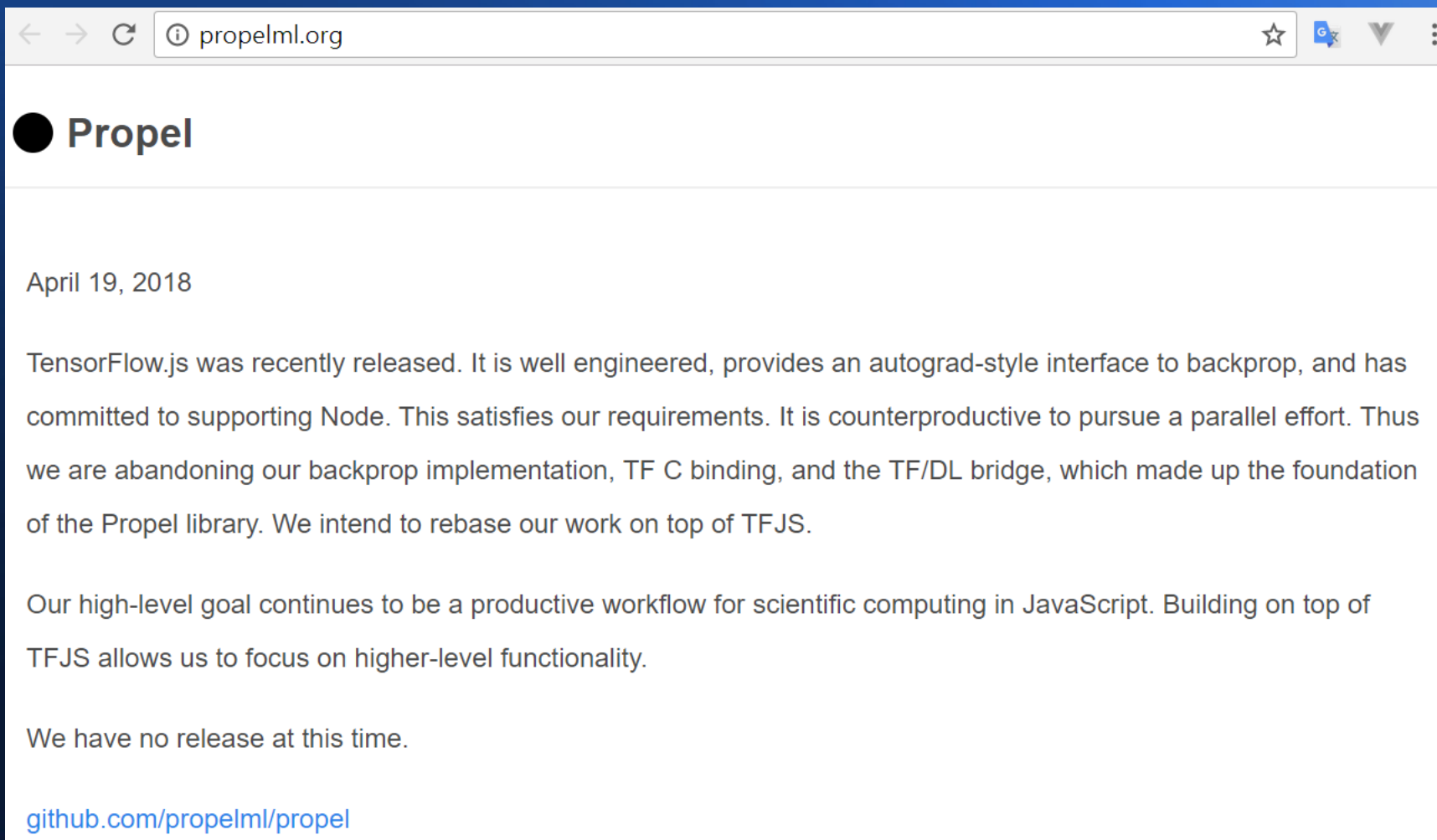
Propel provides a GPU-backed numpy-like infrastructure for scientific computing in JavaScript. JavaScript is a fast, dynamic language which, we think, could act as an ideal workflow for scientific programmers of all sorts.

[API REF](#) [GITHUB](#)

```
import { grad, linspace, plot } from "propel";

f = x => x.tanh();
x = linspace(-4, 4, 200);
plot(x, f(x),
      x, grad(f)(x),
      x, grad(grad(f))(x),
      x, grad(grad(grad(f)))(x),
      x, grad(grad(grad(grad(f)))(x))
```

該專案已經決定轉用 tensorflow.js 放棄了自行打造深度學習套件的想法



所以

- 我接下來應該會更仔細的研究
tensorflow.js 這個套件

並用 tensorflow.js

- 作為我深度學習神經網路的主要工具

這就是我所知道的

- Tensorflow.js 套件

希望你會喜歡

我們下回見！

Bye Bye!